Web Technology II
Note

Name :
Roll No:
Semester:
Contact :
College :

Index

Unit 1

PHP Fundamentals

PHP Introduction:

PHP is a server-side Scripting Language implemented by Rasmus Lerdorf in 1994 using C and Perl languages. He implemented **PHP 1.0** to track total number of visitors in his server. PHP initially stood for Personal Home Page. But now it stands for "PHP: Hypertext Preprocessor". PHP is a widely used, open-source scripting language. PHP scripts are executed on the server.

What is a PHP file?

PHP files are the files that can contain text, HTML, CSS, JavaScript, and PHP code. PHP code is executed on the server, and the result is returned to the browser as plain HTML. PHP files have extension of '.php'.

What can PHP do?

- PHP can generate dynamic page content.
- PHP can create, open, read, write, delete, and close files on the server.
- PHP can collect form data.
- PHP can send and receive cookies.
- PHP can add, delete, modify data in the database.
- PHP can be used to control user-access.
- PHP can encrypt data.

Features of PHP:

The features of PHP are given below:

1. **Simple:** It is simple and easy to learn compared to other scripting languages.
2. **Faster:** It is faster than other scripting languages.
3. **Interpreted:** It is an interpreted language.
4. **Open Source:** It is open Source which means its free to download and use.
5. **Case Sensitive:** PHP is partially case-sensitive language (i.e. Variables names are case-sensitive but keywords/ functions are not).
6. **Platform Independent:** PHP is platform Independent. It runs on any platforms such as Linux, Unix, Mac OS, Windows, etc.
7. **Cross Database:** It supports wide range of databases such as MySQL, Oracle, Microsoft SQL Server, etc.
8. **Security:** It supports different types of security functionalities to apply security to applications.
9. **Flexibility:** PHP code can be embedded with HTML, JavaScript, CSS, XML, etc.
10. **Familiarity:** Most of PHP syntax are inherited from other programming languages like C. So, anyone with programming background can easily understand PHP.
11. **Error Reporting:** PHP have some predefined error reporting constants to generate warning or error notices.
12. **Loosely / Dynamically Typed Language:** PHP supports variable usage without declaring its data type. The type of variable is determined by what type of value is stored in the value at run time. So, it is called Loosely or Dynamically Typed Language.
13. **Cross Web Server:** PHP supports different types of servers like Apache, Tomcat, IIS, etc.
14. **Editor Independent:** We can use any text editor for writing PHP code.

Tools required for developing PHP application:

1. **Web Server:** Software to handle Web Application. Example: Apache, IIS, Nginx, etc.
2. **DBMS:** Oracle, SQL Server, MySQL, SQLite, NoSQL (MongoDB).
3. **PHP:** PHP interpreter or Compiler. *[PHP interpreter is preferred]*
4. **Text Editor / IDE:** Notepad++, Vs Code, etc.

PHP Installation:

1. **Stand-alone Installation:**
   We need to install all these manually:
   - PHP Interpreter
   - Apache Install
   - MySQL Install
   - Text Editor/ IDE Install

2. **Package Installation:**
   We just need to install any one from { XAMPP, WAMP (Windows), LAMP (Linux), MAMP (MacOS) }.

Structure of PHP code:

```
<?php
    // PHP code here…
?>
```

The **Basic Syntax of PHP** is that PHP code always start with "<?php" and ends with "?>" tag. We can start PHP tag as many times as we want in a PHP file. PHP is case sensitive language.

Example: Basic PHP code to display Hello World !! *FileName: index.php*

```
<html>
    <head>
        <title>PHP First Code</title>
    </head>
    <body>
        <h1>
            <?php echo "Hello World !!"; ?>
        </h1>
    </body>
</html>
```

Comments in PHP:

Comments are used to explain the code and make it more readable. PHP interpreter ignores comments, meaning they do not affect the execution of the script. The Types of Comment are:

1. **//** This is Also Single line comment in PHP….
2. **#** This is Single line comment in PHP… *(We can also use # for comment)*
3. **/***
   This is Multi Line
   Comment….
   ***/**

Variables:

Variable is a container that stores the data such as numbers, strings, or arrays. Variables in PHP are dynamic meaning we don't need to declare their data type explicitly. PHP automatically determines their types based on the values assigned. Variables in PHP always start with Dolar Sign ($). The variable name follows immediately after the **$** sign. The rules for declaring variables in PHP are:

1. Variable must start with $ sign.
2. Variable name must begin with a letter or an underscore (_).
3. Variable name can contain letters, numbers, and underscore (_).
4. Variable names are case-sensitive.
5. Reserved keywords cannot be used as variable names.

Example:

```
$name = 'Bishal Bhat';
$age = 23;
$_isBoy = true;

name = 'Hari';                      // Not allowed because '$' missing
$1age = 23;                         // Not allowed because starts with num
$user+name = 'Bishalbhat2002';      // Not allowed because contain '+'
$for = 'BSC-CSIT';                  // Not allowed because contains keyword
```

PHP Data Types:

There are 8 Primary data types in PHP. They are:

1. Integer: Whole numbers (+ve or -ve) without decimals.
2. Float: Numbers with decimal points or exponential notation.
3. String: A sequence of characters enclosed in single (' ') or double (" ") quotes.
4. Boolean: **true** or **false**; true is evaluated as 1 and false is evaluated as empty output.
5. Array: Stores multiple values
6. Object
7. NULL
8. Resource

PHP Operators:

PHP provides a wide range of operators for performing operations on variables and values. The different types of operators in PHP are:

1. **Arithmetic Operators:**
   These operators are used for mathematical calculations.

Operator	Description	Example ($a = 10, $b = 5)	Result
+	Addition	$a + $b	15
–	Subtraction	$a – $b	5
*	Multiplication	$a * $b	50
/	Division	$a / $b	2
%	Modulus (Remainder)	$a % $b	0
**	Exponentiation	$a ** $b *means 10^5*	100,000

2. **Assignment Operators:**
   These operators are used to assign values to variables.

Operator	Example	Equivalent To
=	$a = $b	$a = $b
+=	$a += $b	$a = $a + $b
–=	$a –= $b	$a = $a – $b
*=	$a *= $b	$a = $a * $b
/=	$a /= $b	$a = $a / $b
%=	$a %= $b	$a = $a % $b

3. **Comparison Operators:**

These operators are used to compare 2 values. Their result is always true or false.

Operator	Description	Example ($a = 10, $b = 5)	Result
==	Equal	$a == $b	False
===	Identical (same value & data type)	$a === $b	False
!=	Not Equal	$a != $b	True
<>	Not Equal	$a <> $b	True
!==	Not Identical	$a !===$b	True
<	Less than	$a < $b	False
>	Greater than	$a > $b	True
<=	Less than or Equal to	$a <= $b	False
>=	Greater than or Equal to	$a >= $b	True
<=>	Spaceship Operator (-1, 0, 1)	$a <=> $b	1

<=> operator compares $a and $b. It returns 0 when both have same value, -1 if $a < $b, and 1 if $a > $b.

4. **Logical Operators:**

These operators are used for logical operations.

Operator	Description	Example ($a = true, $b = false)	Result
&&, and	Logical AND	$a && $b	false
||, or	Logical OR	$a || $b	true
!	Logical NOT	!$a	false

5. **Increment Decrement Operators:**

Operator	Description	Example ($a = 5)
++	increment	$a ++, ++$a
– –	decrement	$a – –, – –$a

Both operators can be used as pre-increment and post-increment operators as shown above in example.

6. **String Operators:**

These operators are used for string concatenation.

Operator	Description	Example ($a = ‘hi’, $b = ‘Bishal’)	Result
.	Concatenation	$c = $a . $b	$c = ‘hi Bishal’
.=	Concatenation assignment	$a .= $b	$a = ‘hi Bishal’

7. **Conditional (Ternary Operator):**
   This operator is short form of if-else statement.
   syntax: (condition) ? “valueIfTrue” : “valueIfFalse”;
   Example:

```
$age = 20;
$result = ($age >= 18) ? “Adult” : “Minor”;
```

PHP Expressions:

An **expression** in PHP is anything that evaluates to a value. Expressions are the building blocks of PHP programs, as they define how data is processed and manipulated.

Example:

```
$a = 10;
$b = 5;
$result = $a + $b;                  // 15

$greeting = "Hello" . " World";          // "Hello World"

$count = 5;
$count++;                 // Increments $count by 1 (now 6)
$count--;                 // Decrements $count by 1 (back to 5)
```

Flow Control in PHP:

Flow control statements in PHP determine how code is executed based on conditions or loops. The flow controls are categorized as:

1. **Conditional Statements:**
   a. **if statement**
   b. **if-else statement**
   c. **if-elseif-else statement**
   d. **switch statement**
2. **Looping Statements:**
   a. **for loop**
   b. **while loop**
   c. **do-while loop**
   d. **foreach loop**
3. **Other Flow Control Statements:**
   a. **exit() statement**
   b. **return statement**
   c. **goto statement**

If Statement:

Executes a block of code only if the condition is true.

Example:

```
$age = 20;
if ($age >= 18) {
        echo "You are an adult.";
}
```

If-else Statement:

Executes one block if the condition is true; otherwise, executes another block.

Example:

```
$age = 16;
if ($age >= 18) {
   echo "You are an adult.";
} else {
   echo "You are a minor.";
}
```

If-elseif-else Statement:

Checks multiple conditions in sequence and executes the corresponding block of code when the condition is satisfied. If no condition is satisfied, then it executes the final else block of code.

Example:

```
$marks = 75;
if ($marks >= 90) {
   echo "Grade: A";
} elseif ($marks >= 75) {
   echo "Grade: B";
} elseif ($marks >= 50) {
   echo "Grade: C";
} else {
   echo "Grade: F";
}
```

Switch Statement:

Checks a value against multiple cases and executes the corresponding block of code if condition matched. If no condition is matched, the default block of code is executed.

Example:

```
$day = "Sunday";
switch ($day) {
   case "Sunday":
      echo "Start of the week!";
      break;
   case "Saturday":
      echo "End of the Week";
      break;
   default:
      echo "It's a regular day.";
}
```

For Loop:

Used when the number of iterations is known. It is simple and easy to use than other loops.

Example:

```
for ($i = 1; $i <= 5; $i++) {
   echo "Number: $i <br>";
}
```

While Loop:

Executes a block of code as long as the condition is true. It is called entry-control loop because it checks the condition before each iteration.

Example:

```
$x = 1;
```

```
while ($x <= 5) {
   echo "Number: $x <br>";
   $x++;
}
```

Do while Loop:

Executes the block at least **once**, even if the condition is false. It is called post-control loop or exit control because it checks the condition at the end of each iteration.

Example:

```
$x = 1;
do {
   echo "Number: $x <br>";
   $x++;
} while ($x <= 5);
```

Foreach Loop:

This loop is used to iterate over each element in an array. Using Foreach loop is simpler and easier in array than regular for loop. So, we use foreach loop when we need to iterate over all the array elements.

Example:

```
$colors = ["Red", "Green", "Blue", "Pink", "Yellow"];
foreach ($colors as $color) {
   echo "$color <br>";
}
```

exit() and die() statement:

Both exit() and die() statements terminate script execution immediately. die() is mainly used for error handling statements. exit() is used for normal script termination. They both can display messages. Both are functionally identical in pHP.

Example:

```
echo “Hello World”;
exit(“ Script Stopped here.”);
echo “This will never be displayed…”;
```

Output: Hello world Script Stopped here.

```
$isLogin = false;
if(!$isLogin){
        die(“Access Denied! Please Login.“);
}
echo “Welcome to Dashboard!!”;
```

Output: Access Denied! Please login.

return Statement:

The return statement returns a value to the calling code. It stops the execution of the function when encountered.

Syntax: return value;

Example:

```
function add($a, $b) {
    return $a + $b;                    // Returns the sum
}

$result = add(10, 5);
echo "Sum:  $result";                  // Output: Sum: 15
```

goto Statement:

The goto statement in PHP allows you to jump to another section within the same file.

Syntax:

```
goto label;
...
label:
  // Code to execute after the jump
```

Example:

```
echo "Start<br>";
goto skip;                             // Jump to the label 'skip'
echo "This line will be skipped.<br>";
skip:                                  // This is a label…
echo "Jumped to here!";
```

Including PHP code across Multiple Files:

PHP provides several methods to include code from one file into another, which promotes code reusability and modularity. Some common ways to include PHP code across multiple files are:

1. **include:**
   Inserts the specified file's code at the location of the statement. If the file is not found, a warning is issued, but the script continues execution.
   Syntax: **include "filename.php";**
   Example:

```
<?php
    include "header.php";
    include "footer.php";
    // other code…
?>
```

2. **include_once:**
   Works like include, but ensures that the file is included only once during the execution. This prevents code duplication and redeclaration errors if the file is included multiple times.
   Syntax: **include_once "filename.php";**
   Example:

```
<?php
    include_once "header.php";
    include_once "footer.php";
    // other code…
?>
```

3. **require:**
   Similar to include, but if the file is not found or contains errors, it issues a fatal error and stops the script execution.
   Syntax: **require "filename.php";**
   Example:

```
<?php
    require "dbConnect.php";
    require "functions.php";
?>
```

4. **require_once:**
   Similar to require, but ensures that the file is included only once. If the file is missing or has issues, it stops script execution. This is very useful for configuration files, libraries, or files that must be loaded only once.
   Syntax: **require_once "filename.php";**
   Example:

```
<?php
    require_once "dbConnect.php";
    require_once "functions.php";
?>
```

Different styles of Embedding PHP in Web Pages:

PHP offers several ways to embed PHP code within HTML documents. Here is the most common style with example:

Using Standard PHP tags: (Recommended)
This is most widely used and recommended approach. Works in all PHP configurations. Best for portability and compatibility.
Syntax:

```
<?php
    // php code here….
?>
```

Example: *FileName = greetings.php*

```
<html>
    <head>
        <title>Greetings </title>
    <head>
    <body>
        <?php echo "<h1>Hello World!!</h1>"; ?>
    </body>
</html>
```

Note:
To include, PHP code inside an HTML document, the html documents file extension must be '.php'.

var_dump():

var_dump() is a built-in PHP function used to **display structured information** (type and value) about one or more variables, arrays, or objects.

Syntax: **var_dump($variable1, $variable2, ...);**

Example:

```
<?php
        $str = "Hello";
        $char = 's';
        $num = 67;
        $dec = 65.5;
        $arr = [1, 2, 3];

        var_dump($str, $char, $num, $dec, $arr);
?>
```

Output

```
string(5) "Hello"
string(1) "s"
int(67)
float(65.5)
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(2)
  [2]=>
  int(3)
}
```

Echo and Print in PHP

Both echo and print are used to output data in PHP, but they have slight differences

Echo	Print
It can output one or more strings separated by commas.	It can only output a single string.
It does not return any value.	It returns value 1 if successfully executed.
Slightly faster due to its simplicity.	Slightly slower than echo.

Example:

```
<?php
        echo “Hello World”;                 // Hello World
        print(“Hello World”);               // Hello World

        echo “Hello”, “ ”, “World!”;        // Hello World!
        print(“Hello”, “ ”, “World!”);      // Syntax Error, Can’t display multiple strings..

        $result1 = echo “Hello”;            // Syntax Error, Doest return anything.
        $result2 = print(“Hello”);          // $result2 = 1, Output: Hello
?>
```

PHP code to display multiplication of number 9.

```
<?php
        $num = 9;
        echo "$num Times Multiplication Table:<br>";

        for ($i = 1; $i <= 10; $i++) {
           $result = $num * $i;
           echo "$num * $i = $result<br>";
        }
?
```

***

Unit 2

Function & Strings

Function:

Functions are reusable blocks of code that perform specific tasks. It helps reduce redundancy and makes the code modular and easier to maintain.

Function Declaration:

In PHP, we declare a function using the 'function' keyword followed by the function's name and parentheses. Function names must be unique and are case-insensitive.

Syntax:

```
function functionName(para1, para2, …) {
    // Code to execute
    // return value;
}
```

Function Calling:

After a function is declared, we need to call it to execute the code inside. So, to call a function we simply use the function's name followed by parentheses.

Syntax: functionName();

Example:

```
function sayHello() {
    echo "Hello, Bishal!";
}
sayHello();          //function call, Output: Hello, Bishal!
```

Function Parameters:

We can pass arguments (data) to a function through parameters. Parameters are defined in the parentheses when declaring the function.

Example:

```
function greet($name) {
    echo "Hello, $name!";
}

greet("Bishal");          // Output → Hello, Bishal!
greet("Kirti");           // Output → Hello, Kirti!
```

Returning Values from Function:

A function can also return a value using the **return** statement.

Example:

```
function add($a, $b) {
    return $a + $b;
}

$result = add(5, 3);
echo "Sum: $result";                    // Output→ Sum: 8
echo "Sum: ". add(10, 5);               // Output→ Sum: 15
```

Variable Scopes:

PHP Variables three variable scopes. They are: Local, Global, and Static.

Local Variables:

Local variables are declared inside a function and are only accessible within that function. They are created when the function starts execution and destroyed when the function ends.

Example:

```
function localVariable() {
        $num =  20;                 // Local variable
        echo $num;                  // Output: 20
}

localVariable();                    // Calling the function
echo $num;                          // Error: Undefined variable
```

Global Variables:

Global variables are declared outside of any function, method, or class and can be accessed throughout the script. To access global variables inside a function or class, we must use the **global** keyword or super global Array **$GLOBALS**.

Example: Accessing global variable inside function using **global** keyword:

```
$num = 50;                          // Global variable
function globalVariable() {
        global $num;        // Access the global variable using global keyword first
        echo $num;          // Then only use the global variable. Output: 50
}
globalVariable();
```

Example: Accessing global variable inside function using $**GLOBALS** super global Array:

```
$num = 100;                                 // Global variable
function globalVariable() {
        echo $GLOBALS['num'];               // Output: 100
}
globalVariable();                           // Calling the function
```

Static Variables:

Static variables are declared inside a function using **static** keyword. They retain their values across multiple calls to that function. Unlike local variables, static variables are not destroyed when the function ends. Static variables only exist within the function scope but maintains their state.

Example:

```
function countCall() {
   static $count = 0;                   // Static variable
   $count++;
   echo "Function called $count time(s)\n";
}
countCall();                    // Output: Function called 1 time(s)
countCall();                    // Output: Function called 2 time(s)
countCall();                    // Output: Function called 3 time(s)
```

Note: static $count; and static $count = 0; Both statements are same. We can set to static variable to any value. But if we don't give any value during declaration, it is automatically set to 0.

Variable function:

In PHP we can call a function using a variable. In variable function we store the name of a function in a variable as a string and then call the function using that variable.

Syntax:

```
function functionName($para1, $para2, … ) {   // code to execute….   }
$variableName = 'functionName';
$VariableName();
```

Example:

```
function sayHello($name) {
        echo "Hello $name…";
}

$variable = 'sayHello';
$variable('Bishal');            // Calls to sayHello() with 'Bishal' argument
                                // Output: Hello Bishal…
```

Anonymous Function (Closure):

An anonymous function, also known as a closure, is a function without a name. They are declared using the **function** keyword without providing a name. They can be stored in variables, returned by other functions, or passed as arguments to other functions.

Syntax:

```
function($para1, $para2, … ) { // Code to Execute … }
```

Example:

```
$sum = function($a, $b) {
          echo "Sum: ". $a+$b;
};
$sum(10, 20);                   // Output → 30, calls the anonymous function
```

Default Argument Function:

A default argument function allows you to specify default values for function parameters. When the function is called without any argument, the default value is used. This feature makes functions more flexible and reduces the need for overloading in some cases.

Syntax:

```
function functionName($param1 = defaultValue) {  // Function code …  }
```

Example:

```
function greet($name = "Guest") {
        echo "Hello, $name!";
}

greet();                        // Output: Hello, Guest!
greet("Bishal");                // Output: Hello, Bishal!
```

Some rules for Default arguments:

- The parameters with default values must be on the right side. I.e. Non default valued parameters must be on left side. **Example:** function ($a, $b, $c = 10, $d = 50){ // code }
- We can use any data type value as default value.

String in PHP:

String is a sequence of characters. PHP provides a wide range of functionality to work with strings, from simple operations like concatenation to advanced functions such as pattern matching with regular expressions. The types of strings in PHP are:

1. **Single quoted String (‘ ’):**
   It treats everything inside as plain text. The variables inside **single quotes** are not replaced with their values. It only supports \\ (backslash) and \' (single quote).
   Example:

```
$name = "Bishal";
echo 'Hello, $name!';          // Output: Hello, $name!
echo 'It\'s a great day!';     // Output: It's a great day!
```

2. **Double quoted string (“ ”):**
   Variables inside **double quotes** are replaced with their values. It supports many escape sequences like \n, \t, \", etc.
   Example:

```
$name = "Bishal";
echo "Hello, $name!";          // Output: Hello, Bishal!
echo "Line1\nLine2";           // Output: Line1
                               // Line2
```

Escape Sequences:

Escape sequences are special character combinations that allow you to insert special characters inside double-quoted (") or single-quoted (') strings in PHP. Some common Escape sequences in PHP are:

Escape Sequence	Meaning	Works in Single (') Quotes?	Works in Double (") Quotes?
\'	Single quote (')	Yes	Yes
\"	Double quote (")	No	Yes
\\	Backslash (\)	Yes	Yes
\n	Newline (Line Break)	No	Yes
\r	Carriage Return	No	Yes
\t	Tab Space	No	Yes
\$	Dollar sign ($)	No	Yes

Example:

```
echo "Hello \"PHP\"!\n";        // Output: Hello "PHP"!    (with a new line)
echo "Tab\tSpace";              // Output: Tab    Space
```

Printing Strings in PHP:

We can display/ print strings in PHP in following ways.

```
echo "This is a string";        // This is a string          (preferred one)
print "This is also a string";  // This is also a string
```

String Functions:

PHP provides built-in functions to manipulate, search, format, and analyze strings. The most common string functions are:

1. **strlen():**
   This function returns the number of characters in a string, including spaces.
   Example: echo strlen("Hello"); // Output: 5

2. **strtolower() and strtoupper():**
   These functions return a new string by converts all characters in a string to lowercase or uppercase respectively.
   Example: echo strtolower("HELLO"); // Output: "hello"
   echo strtoupper("hello"); // Output: "HELLO"

3. **trim():**
   This function returns a new string by removing all whitespace from both sides (starting and ending) of a string.
   Example: echo trim(" Hello "); // Output: "Hello"

4. **strrev():**
   This function returns a new string by reversing a string, making the first character last and vice versa.
   Example: echo strrev("Hello"); // Output: "olleH"

5. **strcmp():**
   This function compares two strings case-sensitively. Returns 0 if equal, positive if first is greater, negative if smaller.
   Example: echo strcmp("apple", "apple"); // Output: 0
   echo strcmp("Apple", "apple"); // Output: -1

6. **ucfirst() and ucwords():**
   These functions return a new string by capitalizing the first character of a string (ucfirst) or first character of each word (ucwords).
   Example: echo ucfirst("hello"); // Output: "Hello"
   echo ucwords("hello world"); // Output: "Hello World"

7. **str_replace()**
   This function returns a new string by replacing all occurrences of a search string with a replacement string.
   Syntax: str_replace('oldValue', 'newValue', 'string');
   Example: echo str_replace("world", "PHP", "Hello world"); // Output: "Hello PHP"

8. **strpos():**
   This function finds the position of the first occurrence of a substring in a string.
   Syntax: strops('String', 'substring');
   Example: echo strpos("Hello world", "world"); // Output: 6

9. **substr():**
   This function returns a portion of a string specified by start and length parameters.
   Syntax: substr('string', startIndex, length);
   Example: echo substr("Hello world", 6, 5); // Output: "world"

10. **explode():**
    This function returns an array by splitting a string into an array by a specified delimiter. This function returns an array.
    Syntax: $array = explode('delimiter', 'string');
    Example:

```
print_r(explode(" ", "Hello world"));
                                        // Output: Array([0]=>"Hello", [1]=>"world")
```

    The **print_r()** function is used here instead of echo because explode() function returns an array. The contents of array cannot be displayed directly using echo. so, to make our code short and sweet, we used print_r() function. print_r() prints the content of an array or object in a human readable format.

11. **implode():**
    This function returns string by Joining all array elements into a string with a specified separator. Separator is used to join all the array elements i.e. Separator is placed between each array elements when converted to string. This function also returns an string.
    Syntax: $string = implode('separator', $array);
    Example:

```
echo implode("-", ["2023", "12", "31"]);          // Output: "2023-12-31"
echo implode(", ", ["Cricket", "Football", "Basketball"]);
                                    // Output: "Cricket, Football, Basketball"
```

12. **htmlspecialchars():**
    This function returns a string by converting special characters to HTML entities to prevent XSS attacks.
    Example:

```
echo htmlspecialchars("<script>");          // Output: "&lt;script&gt;"
```

13. **number_format():**
    This function returns a new string by formatting number with grouped thousands and decimal points.
    Example:

```
echo number_format(1234567);          // Output: "1,234,567"
```

PHP Regular Expressions:

A **Regular Expression (RegEx)** is a pattern used to match and manipulate strings. The basic syntax of regular expression is

```
$regex = "/pattern/modifiers";
```

Pattern: The sequence of characters to be matched within the string.

Modifiers: They are optional flags to control the pattern's behavior. The modifiers in PHP regular expressions are:

Modifier	Description
i	Case-insensitive match
m	Multi-line mode (treats ^ and $ per line)
u	Enables UTF-8 matching

Meta Characters in PHP Regular Expression:

Meta characters are special characters with specific meanings. They are used to define the pattern for regular expression. The meta characters are:

Metacharacter	Meaning
^	Matches the **beginning** of a string
$	Matches the **end** of a string
. *(dot)*	Matches **any single character** except newline
*	Matches **0 or more** occurrences
+	Matches **1 or more** occurrences
?	Matches **0 or 1** occurrence (optional)
{n}	Matches **exactly n** occurrences
{n,}	Matches **at least n** occurrences
{n,m}	Matches **between n and m** occurrences
\	Escapes a metacharacter [**Example:** '\.' matches dot(.) in string]
\d	Matches any digit (equivalent to [0-9])
\D	Matches any non-digit character.
\w	Matches any word character (equivalent to [A-Za-z0-9_])
\W	Matches any non-word character (equivalent to [^A-Za-z0-9_])
\s	Matches any whitespace character.
\S	Matches any non-whitespace character.
^abc	Matches "abc" at the start of a string.
abc$	Matches "abc" at the end of a string.
[abc]	Matches any character inside bracket.
[^abc]	Matches any character NOT in bracket.

PHP functions for Regular Expressions:

1. **preg_match():**

   It searches for a **pattern in a string** and **returns 1 if found, 0 if not found**. It stops searching after the first match.

   Syntax: preg_match(pattern, string, matches);

   pattern: The regular expression pattern.
   string: String to search in
   matches (optional): Stores the matched string. It is an array.

   Examples:

```
$pattern = "/hello/i";                    // Case-insensitive match
$string = "Hello, World!";
if (preg_match($pattern, $string)) {
   echo "Match found!";
} else {
   echo "No match.";
}
```

   Output: Match found

```
$pattern = "/\d+/";                       // Pattern to match numbers
$string = "The price is 250 dollars.";
preg_match($pattern, $string, $match);
echo $match[0];                           // Output: 250
```

2. **preg_match_all():**
   Unlike preg_match(), which **stops after the first match**, preg_match_all() finds **all** matches in a string. Stores **all matched values** in an array. This function returns number of times the pattern matched within the string. If no match found, it returns 0.
   Syntax: preg_match_all(pattern, subject, matches);

   Example:

```
$pattern = "/\d/";                                    // To match digits [0-9]
$string = "I have 2 apples, 4 oranges, and 6 bananas.";
preg_match_all($pattern, $string, $matches);
print_r($matches);                                    // To display array
```

   Output:

```
Array ( [0] => Array ( [0] => 2 [1] => 4 [2] => 6 ) )
```

3. **preg_replace():**
   This function returns a new string by searching for a **pattern** and replacing it with specified string. It doesn't change the original string.
   Syntax: preg_replace(pattern, replacement, string, limit, count);
   replacement: The replacing string.
   limit: Optional, this limits the number of replacements.
   count: Optional, this stores the number of replacements made.

   Example:

```
$pattern = "/apple/i";
$string = "I like Apple.";
$replacement = "Banana";
$newString = preg_replace($pattern, $replacement, $string);

echo $newString;                                      // I like Banana.
echo $string;                                         // I like Apple.
```

***

Unit 3

Arrays and Objects

Array:

An array is a data structure that stores multiple values in a **single variable**. It allows storing and managing multiple elements under one name. We can store different type of values in a same array unlike C/C++.

Syntax for Creating Array:

1. $arrayName = array(value1, value2, value3, …); // using array() function
2. $arrayName = [value1, value2, value3, …]; // using square brackets

Example:

```
$names = ['Bishal', 'Kirti', 'Jethalal', 'Babita'];
$mixedArr = ['karan', true, 100, 40.5, 'sonu'];
```

Types of Arrays in PHP:

In PHP, there are 3 types of arrays. They are:

1. Indexed (Numerical) Array
2. Associative Array
3. Multidimensional Array

Indexed Array:

An **indexed array** is a collection of elements where each value is stored with an automatically assigned **numeric index** (starting from 0).

Example:

```
$fruits = ['Apple', 'Banana', 'Mango'];
```

Accessing Elements:

We can access any element of indexed array using index of that elements as:

```
echo $fruits[0];        // Output: Apple
echo $fruits[1];        // Output: Banana
```

Modifying an Element:

We can modify any element of indexed array using index of that element as:

```
$fruits[1] = "Orange";
print_r($fruits);       // print_r() function used to display array directly.
                        // Output: Array ( [0] => Apple [1] => Orange [2] => Mango )
```

Adding Elements:

We can add new element at the end of an array as:

```
$fruits[] = "Pineapple";        // Adds a new element at the end
```

Looping Through an Indexed Array:

```
foreach ($fruits as $fruit) {
    echo $fruit . " ";
}
                                // Output: Apple Orange Mango Pineapple
```

Associative Array:

An **associative array** allows us to store data in the form of **named key-value pairs**. This array uses keys instead of numeric indexes. It is useful for storing structured data.

Syntax:

$arrayName = ['key1' => 'value1', 'key2' => 'value2', 'key3' => 'value3', …];

Example:

Sstudent = ['name' => 'Bishal Bhat', 'age' => '23', 'course' => 'BSC-CSIT'];

Accessing Elements:

We access the elements in Associative array using key names as.

```
echo $student["name"];          // Output: Bishal Bhat
echo $student["age"];           // Output: 23
```

Modifying Values:

We modify the elements in Associative array using key names as.

```
$student["age"] = 24;           // Changing age from 23 to 24
```

Adding New key-Value Pairs:

We can add new elements in Associative array as:

```
$student['email'] = 'bishalbhat3313@gmail.com';
```

Looping Through Associative Array:

```
foreach ($student as $key => $value){
        echo "$key: $value\n";
}
```

```
name: Bishal Bhat
age: 23
course: BSC-CSIT
```

Multidimensional Array:

An array containing another array is called multidimensional array. It is used for tabular data or hierarchical structures. It supports nested indexing for element access.

1. **Indexed Multidimensional Array:**

   Example:

```
$students = [
        ["Bishal", 23, "BSC-CSIT"],
        ["Ram", 24, "BBA"],
        ["Hari", 22, "BBS"]
];
```

Accessing Elements:

```
echo $students[0][0];           // Output: Bishal
echo $students[1][2];           // Output: 24
```

Looping through an Indexed Multidimensional Array:

```
foreach ($students as $student) {
    echo "Name: $student[0]  Age: $student[1]  Course: $student[2] \n";
}
```

```
Name: Bishal  Age: 23  Course: BSC-CSIT
Name: Ram  Age: 24  Course: BBA
Name: Hari  Age: 22  Course: BBS
```

2. **Associative Multidimensional Array:**

Example:

```
$students = [
        "Bishal" => ["age" => 23, "course" => "BSC-CSIT"],
        "Ram" => ["age" => 24, "course" => "BBA"],
        "Hari" => ["age" => 22, "course" => "BBS"]
];
```

Accessing Elements:

```
echo $students["Bishal"]["course"];        // Output: BSC-CSIT
echo $students["Ram"]["Age"];              // Output: 24
```

Looping Through an Associative Multidimensional Array:

```
foreach ($students as $name => $info) {
        echo "Name: $name  Age: {$info['age']}  Course: {$info['course']}\n";
}
```

```
Name: Bishal  Age: 23  Course: BSC-CSIT
Name: Ram  Age: 24  Course: BBA
Name: Hari  Age: 22  Course: BBS
```

Notice Something?

While working with arrays and objects inside double inverted comma, PHP parser cannot safely distinguish between array Names and key Names. So, accessing the array elements with this line:

echo "Name: $name Age: $info['age'] Course: $info['course']\n"; // shows error.

To avoid the error, we can rewrite above statement as:

echo "Name: $name, Age: " . $info['age'] . ", Course: " . $info['course'] . "\n";

But above statement also looks abit clumsy so, we use **'{}'** with arrayNames inside **"..."**as:

echo "Name: $name Age: {$info['age']} Course: {$info['course']}\n";

The **{}** tells PHP parser exactly where the variable starts and ends. Just use **{}** with array names while working with arrays and objects.

We can use any one statement from the last 2. But I prefer last one.

Array Functions:

The Array Functions are given below:

1. **array_push():** Adds one or more elements to the end of an array.

```
$fruits = ["Apple", "Banana"];
array_push($fruits, "Cherry");                    // Adds elements to the end
print_r($fruits);          // Output: Array ( [0] => Apple [1] => Banana [2] => Cherry)
```

2. **array_pop():** Removes the last element from an array and returns it.

```
$fruits = ["Apple", "Banana"];
$lastFruit = array_pop($fruits);                  // Removes the last element
echo $lastFruit;                                  // Output: Banana
```

3. **array_unshift():** Adds one or more elements to the beginning of an array.

```
$fruits = ["Banana", "Cherry"];
array_unshift($fruits, "Apple");                    // Adds "Apple" at the beginning
print_r($fruits);          // Output: Array ( [0] => Apple [1] => Banana [2] => Cherry )
```

4. **array_shift():** Removes the first element from an array and returns it.

```
$fruits = ["Apple", "Banana", "Cherry"];
$firstFruit = array_shift($fruits);          // Removes the first element
echo $firstFruit;                            // Output: Apple
```

5. **sort():** Sorts indexed arrays in ascending order.

```
$numbers = [4, 2, 8, 1];
sort($numbers);                              // Sorts in ascending order
print_r($numbers);          // Output: Array ( [0] => 1 [1] => 2 [2] => 4 [3] => 8 )
```

6. **rsort():** Sorts indexed arrays in descending order.

```
$numbers = [4, 2, 8, 1];
rsort($numbers);                                  // Sorts in descending order
print_r($numbers);          // Output: Array ( [0] => 8 [1] => 4 [2] => 2 [3] => 1 )
```

7. **asort():** Sorts associative arrays by values while maintaining keys.

```
$ages = ["Bishal" => 23, "Anita" => 22, "Suresh" => 30];
asort($ages);                                  // Sorts values in ascending order
print_r($ages);          // Output: Array ( [Anita] => 22 [Bishal] => 23 [Suresh] => 30 )
```

8. **ksort():** Sorts associative arrays by keys.

```
$ages = ["Bishal" => 23, "Anita" => 22, "Suresh" => 30];
ksort($ages);                                // Sorts keys in ascending order
print_r($ages);          // Output: Array ( [Anita] => 22 [Bishal] => 25 [Suresh] => 30 )
```

9. **array_merge():** Merges two or more arrays into one array.

```
$colors1 = ["Red", "Green"];
$colors2 = ["Blue", "Yellow"];
$result = array_merge($colors1, $colors2);
print_r($result);
                    // Output: Array ( [0] => Red [1] => Green [2] => Blue [3] => Yellow )
```

10. **array_combine():** Combines two arrays (keys from the first, values from the second).

```
$keys = ["name", "age"];
$values = ["Bishal", 23];
$combined = array_combine($keys, $values);
print_r($combined);                  // Output: Array ( [name] => Bishal [age] => 23 )
```

11. **array_slice():** Extracts a portion of an array.

```
$numbers = [1, 2, 3, 4, 5];
$slice = array_slice($numbers, 1, 2);   // Extracts elements from start index 1 and length 2
print_r($slice);                                  // Output: Array ( [0] => 2 [1] => 3 )
```

Syntax: $subArray = array_slice($arrayName, startIndex, elementsToExtract);

12. in_array(): Checks if a value exists in an array. It returns true if value exists in an array. Otherwise returns false.

```
$fruits = ["Apple", "Banana", "Cherry"];
if(in_array("Banana", $fruits))
        echo "Exists!";
else
        echo "Doesn't exist!";
                                                        // Output: Exists!
```

13. array_search(): Searches the array for a value and returns the first matched elements index.

```
$fruits = ["Apple", "Banana", "Cherry"];
$key = array_search("Cherry", $fruits);
echo $key;                                              // Output: 2
```

14. array_filter(): Filters an array using a callback function.

```
$numbers = [1, 2, 3, 4, 5];
$evenNumbers = array_filter($numbers, function($num) {
   return $num % 2 == 0;                                // returns even numbers
});
print_r($evenNumbers);                  // Output: Array ( [1] => 2 [3] => 4 )
```

15. array_map(): Applies a callback function to each element of an array.

```
$numbers = [1, 2, 3];
$squared = array_map(function($num) {
      return $num * $num;
}, $numbers);
print_r($squared);                      // Output: Array ( [0] => 1 [1] => 4 [2] => 9 )
```

16. array_keys(): Returns all keys of an array.

```
$person = ["name" => "Bishal", "age" => 23];
$keys = array_keys($person);
print_r($keys);                         // Output: Array ( [0] => name [1] => age )
```

17. array_values(): Returns all values of an array.

```
$values = array_values($person);
print_r($values);                       // Output: Array ( [0] => Bishal [1] => 23 )
```

18. array_diff(): Returns all the non-matching elements from first array.

```
$array1 = ["Red", "Green", "Blue"];
$array2 = ["Green", "Yellow"];
$difference = array_diff($array1, $array2);
print_r($difference);                   // Output: Array ( [0] => Red [2] => Blue )
```

19. array_intersect(): Returns the matching elements from both arrays.

```
$array1 = ["Red", "Green", "Blue"];
$array2 = ["Green", "Yellow"];
$intersection = array_intersect($array1, $array2);
print_r($intersection);                 // Output: Array ( [1] => Green )
```

20. count(): Counts the number of elements in an array.

```
$numbers = [1, 2, 3];
echo count($numbers);                                    // Output: 3
```

21. array_unique(): Returns a new array by removing the duplicate values.

```
$numbers = [1, 2, 3, 2, 1];
$unique = array_unique($numbers);
print_r($unique);                          // Output: Array ( [0] => 1 [1] => 2 [2] => 3 )
```

22. array_reverse(): Returns a new array by Reversing the order of elements.

```
$numbers = [1, 2, 3, 14, 7];
$reversed = array_reverse($numbers);
print_r($reversed);              // Array ( [0] => 7 [1] => 14 [2] => 3 [3] => 2 [4] => 1 )
```

23. implode():

This function returns string by Joining all array elements into a string with a specified separator. Separator is used to join all the array elements i.e. Separator is placed between each array elements when converted to string. This function also returns an string.

Syntax: $string = implode('separator', $array);

Example:

```
echo implode("-", ["2023", "12", "31"]);                    // Output: "2023-12-31"
echo implode(", ", ["Cricket", "Football", "Basketball"]);
                                                  // Output: "Cricket, Football, Basketball"
```

Converting Array into Variables:

The **extract()** function takes an associative array and converts its keys into variable names and their corresponding values into the values of those variables.

Example:

```
# Associative array
$data = [
   "name" => "Bishal",
   "age" => 23,
   "course" => "BSC-CSIT"
];

extract($data);                          // Convert array keys to variables

# Access the variables
echo $name;                                  // Output: Bishal
echo $age;                                   // Output: 23
echo $course;                                // Output: BSC-CSIT
```

Note:

We should be cautious when using extract(), as it can overwrite existing variables in our script.

Converting Variables to an Array:

The **compact()** function creates an associative array from a list of variable names. The variable names become the keys, and their values become the corresponding values in the array.

Example:

```
# Define variables
$name = "Bishal";
$age = 23;
$course = "BSC-CSIT";

$data = compact("name", "age", "course");          // Convert variables into an array

print_r($data);
                                    // Output: Array ( [name] => Bishal [age] => 23 [course] => BSC-CSIT )
```

Traversing an Array:

Traversing an array in PHP means iterating through its elements to access or manipulate them. We can traverse an array using loops (for loop, foreach loop, while loop, do while loop), array_map(), array_keys().

Syntax for Loops:

```
# foeach loop for Indexed Array:
foreach($array as $element){
        // code to execute..
}

# foeach loop for Associative Array:
foreach($array as $key => $value){
        // code to execute..
}

# for loop for Indexed Array:
for($i = 0; $i < count($array); $i++){
        // code to execute ..
}
```

Example:

```
$numbers = [1, 2, 3, 4, 5];
for ($i = 0; $i < count($numbers); $i++) {
   echo $numbers[$i] . "\n";                                    // Output: 1 2 3 4 5
}
```

Class:

A class in PHP is a blueprint for creating objects. It defines properties (variables) and methods (functions) that the objects of the class will have. Classes are fundamental to object-oriented programming (OOP) in PHP, which allows developers to structure and organize code efficiently.

Declaring a class:

A class is declared using the **class** keyword, followed by the class name and curly braces **{}**.

Syntax:

```
class className {
        //properties & Methods
}
```

Properties:
Properties are variables defined inside a class. They represent the attributes of the class. Public properties can be accessed directly using object of that class but for accessing private properties we must use methods. Properties have 3 access modifiers. Access modifiers define accessibility of the property. The 3 access modifiers are:

1. **Public:** Accessible from anywhere using the class object.
2. **Private:** Accessible only within the class. Cannot be accessed outside the class.
3. **Protected:** Accessible within the class and its subclasses (inherited classes).

Methods:
Methods are functions defined inside a class. They represent the behavior of the class.

Objects:
An object is an instance of a class. Objects are the actual entities created based on that class. They have properties (variables) and methods (functions) defined with in the class. To create object we use **new** keyword and class Name. The syntax is given as:

$objectName = new className();

Example:

```
# Declaring class
class Car {
        # Properties
        public $brand;
        public $color;
        private $price;

        # Methods
        public function setPrice($price){
         $this -> price = $price;
        }
        public function drive() {
           echo "The car is driving!";
        }
}

# Creating an Object and accessing public properties
$car = new Car();                          // Create an object of the Car class
$car -> brand = "BMW";                     // Set the public property directly
$car -> color = "Red";                      // Set property
$car -> setPrice(100000);                  // Calls method to set private property.
$car -> drive();                           // Calls method
                                           // Output: The car is driving!
```

Constructor:

A constructor is a method that is automatically called when an object of the class is created. It is typically used to initialize object properties or perform setup tasks. Unlike other programming language, where constructor name is same as class name. In PHP, one class can have only one constructor. In PHP, constructor name is fixed "_ _construct()". ['2 underscore' and then 'construct()']

Syntax: public function __construct ($para1, $para2, …) { // code to execute }

Types of Constructor:

1. **Default Constructor:**
   A constructor without any parameters which initializes default values for the object's properties is called default constructor.
   Syntax: public function __construct () { // code to execute }

2. **Parameterized Constructor:**
   A constructor that accepts parameters to initialize object properties with specific values.
   Syntax: public function __construct ($para1, $para2, …) { // code to execute }

Example:

```
<?php
    class Users {
        private $name;

        public function __construct($name = "Guest") {
            $this->name = $name;
            echo "Object Created.\n";
            $this->greet();
        }

        # Function to greet the user
        public function greet() {
            echo "Hello, $this->name\n";
        }
    }

    # Creating objects
    $user1 = new Users();                    // Uses default "Guest"
    $user2 = new Users("Bishal");            // Uses "Bishal"
?>
```

Output:

```
Object Created.
Hello, Guest
Object Created.
Hello, Bishal
```

Destructor:

A destructor is a method that is automatically called when an object is destroyed or goes out of scope. It's often used for cleanup tasks, such as releasing resources or closing connections. The name of destructor method is also fixed to "__destruct()".

Syntax: **public function __destruct() { // code to execute… }**

Example:

```
<?php
    class MyClass {
        public function __construct() {
            echo "Object created.\n";
        }

        public function __destruct() {
            echo "Object destroyed.\n";
        }
    }

    $obj = new MyClass();
?>
```

Output

```
object created.
Object destroyed.
```

Inheritance:

Inheritance is a fundamental concept in Object-Oriented Programming (OOP) that allows a class (child class) to acquire properties and methods from another class (parent class). This helps in code reusability and organization. In PHP, we use **extends** keyword for inheritance.

Syntax:

class childClass **extends** parentClass { // code for child class}

Types of Inheritance:

1. **Single Inheritance:**

   A child class inherits from only one parent class. The child class gets access to all public and protected properties and methods of the parent class.

   Example:

```
<?php
    class ParentClass {
        public function greet() {
            echo "Hello from Parent Class\n";
        }
    }

    class ChildClass extends ParentClass {
        public function childGreet() {
            echo "Hello from Child Class\n";
        }
    }

    $child = new ChildClass();
    $child -> greet();              // Outputs: Hello from Parent Class
    $child -> childGreet();         // Outputs: Hello from Child Class
?>
```

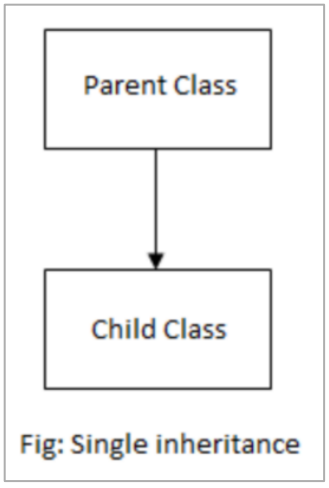


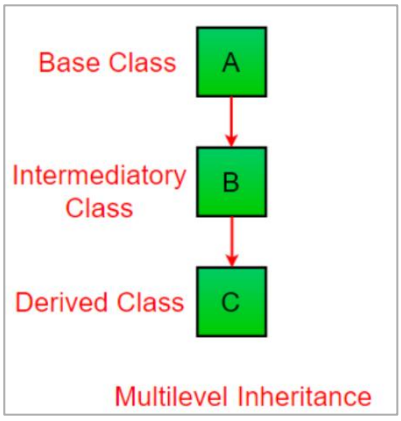
Fig: Single inheritance

2. **Multi-level Inheritance:**

   A class inherits from another class, which itself is inherited from another class. This forms a chain of inheritance.

   Example:

```
<?php
    class GrandParent {
        public function sayHi() {
            echo "Hi from Grandparent \n";
        }
    }

    class ParentClass extends GrandParent {
        public function sayHello() {
            echo "Hello from Parent Class\n";
        }
    }

    class ChildClass extends ParentClass {
        public function sayWelcome() {
            echo "Welcome from Child Class\n";
        }
    }
```



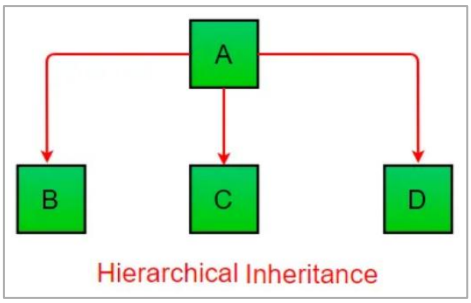
```
$child = new ChildClass();                    // Object create
$child->sayHi();                    // Outputs: Hi from Grandparent
$child->sayHello();                  // Outputs: Hello from Parent Class
$child->sayWelcome();               // Outputs: Welcome from Child Class
?>
```

3. **Hierarchical Inheritance:**
   In hierarchical inheritance, multiple child classes inherit from a single parent class.
   Example:

```
<?php
    class Vehicle {
        public function start() {
            echo "Vehicle starting...\n";
        }
    }

    class Car extends Vehicle {
        public function drive() {
            echo "Car driving...\n";
        }
    }

    class Bike extends Vehicle {
        public function ride() {
            echo "Bike riding...\n";
        }
    }

    $car = new Car();                    // Object created
    $car->start();                       // Inherited method
    $car->drive();                       // Car-specific method

    $bike = new Bike();                  // Object created
    $bike->start();                      // Inherited method
    $bike->ride();                       // Bike-specific method
?>
```

Output:

```
Vehicle starting...
Car driving...
Vehicle starting...
Bike riding...
```



4. **Multiple Inheritance (Via Interfaces):**
   PHP does not support multiple inheritance (i.e., a class cannot extend multiple classes). But we can use **Interfaces** to achieve similar functionality as multiple inheritance. The multiple inheritance through interface is explained as:

PHP Interfaces:

Interfaces are declared using **interface** keyword. Interfaces only contains method declarations (no method bodies). The methods must be public (i.e. they cannot be private or protected). A class implements an interface using **implements** keyword. A class can implement multiple interfaces, enabling behavior similar to multiple inheritance. A class that implements interfaces must define all the methods of the implemented interfaces inside it.

Syntax: **interface** interfaceName { // public interface function declarations… }

Example:

```
<?php
    # Base class
    class Animal {
        public function eat() {
            echo "Eating food.\n";
        }
    }

    # Interface for flying
    interface Flyable {
        public function fly();
    }

    # Interface for swimming
    interface Swimable {
        public function swim();
    }

    # Class extending Animal and implementing interfaces
    class Bird extends Animal implements Flyable, Swimable {
        public function fly() {
            echo "The bird is flying.\n";
        }

        public function swim() {
            echo "The bird is swimming.\n";
        }
    }

    $bird = new Bird();
    $bird -> eat();          // Outputs: Eating food.
    $bird -> fly();          // Outputs: The bird is flying.
    $bird -> swim();         // Outputs: The bird is swimming.
?>
```

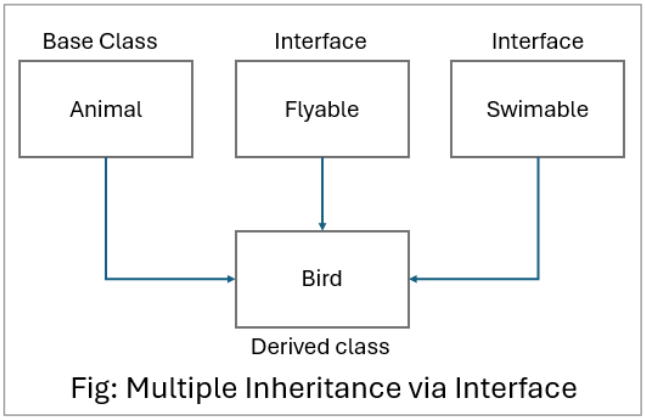


Fig: Multiple Inheritance via Interface

Output:

```
Eating food.
The bird is flying.
The bird is swimming.
```

Abstract Classes:

An **abstract class** in PHP is a class whose object cannot be created. It is meant to be **extended** by other classes and can contain both **abstract methods** (methods without a body) and **concrete methods** (methods with a body). Abstract class is declared using the **abstract** keyword. Abstract methods are declared using **abstracted** keyword in abstract class. And abstract methods are defined in derived class. Abstract class supports **constructors**, **properties**, and **method implementations**.

Example:

```
<?php
    # Abstract class
    abstract class Animal {
        # Abstract method (no implementation)
        abstract public function makeSound();

        # Concrete method (with implementation)
        public function eat() {
            echo "This animal is eating.\n";
        }
    }

    #Subclass extending the abstract class
    class Dog extends Animal {
        # Implementing the abstract method
        public function makeSound() {
            echo "Woof! Woof!\n";
        }
    }

    # Subclass extending the abstract class
    class Cat extends Animal {
        # Implementing the abstract method
        public function makeSound() {
            echo "Meow! Meow!\n";
        }
    }

    # Creating objects of the subclasses
    $dog = new Dog();                   // Object created
    $dog -> makeSound();                // Outputs: Woof! Woof!
    $dog -> eat();                       // Outputs: This animal is eating.

    $cat = new Cat();                   // Object created
    $cat -> makeSound();                // Outputs: Meow! Meow!
    $cat -> eat();                       // Outputs: This animal is eating.
?>
```

Static Variable with Class:

A **static variable** in a class is **shared across all instances (objects) of that class**. It belongs to the class itself rather than any specific object. It is declared using the **static** keyword. It does not reset for each object (maintains value across all instances). To access this variable inside class we use **self::$variableName**. For accessing outside the class we use **ClassName::$variableName.**

Example:

```
<?php
        class Counter {
          public static $count = 0;                    // Static variable

          public function __construct() {
                    self::$count++;                    // Increment count for every object created
                    echo "Object created. Count = " . self::$count . "\n";
          }
        }

        # Creating objects
        $obj1 = new Counter(); // Count = 1
        $obj2 = new Counter(); // Count = 2
        $obj3 = new Counter(); // Count = 3

        # Accessing static variable without creating an object
        echo "Total Objects Created: " . Counter::$count . "\n";
?>
```

Output:

```
Object created. Count = 1
Object created. Count = 2
Object created. Count = 3
Total Objects Created: 3
```

***

Unit 4

Form Processing

HTTP Basics:

HTTP (Hyper Text Transfer Protocol) is the protocol used for transferring data (like web pages, images, videos) over the **web**. When we open a website, our browser uses HTTP to request and receive information from the server. Common HTTP Methods are given below:

Method	Description
GET	Request data from the server (read)
POST	Send data to the server (submit)
PUT	Update existing data on the server
DELETE	Remove data from the server

Some Common HTTP Status Codes are:

Code	Meaning
200	OK (success)
404	Not Found – Page or resource not found
500	Internal Server Error – Generic server error
403	Forbidden – Access Denied
301	Moved Permanently – Resource has new URL

Server Variables:

Server variables are the predefined variables that contains information about the server, current request, headers, environment, etc. These variables are stored in a special super global array called **$_SERVER.** To access server variables, we use the given syntax:

```
$_SERVER['variable_name']
```

There are many server variables. Some are given as:

Variable	Description
$_SERVER['PHP_SELF']	The file name of the currently executing script.
$_SERVER['SERVER_NAME']	Name of the Server (Ex: localhost).
$_SERVER['HTTP_HOST']	Host header from the current request (Ex: localhost or domain)
$_SERVER['REQUEST_METHOD']	Method used to access the page (GET, POST)
$_SERVER['REQUEST_URI']	URI that was given to access the page.
$_SERVER['REMOTE_ADDR']	IP address of the user.
$_SERVER['SERVER_SOFTWARE']	Name and version of web server software (Ex: Apache/ 2.4.54)
$_SERVER['SERVER_PROTOCOL']	Protocol and version used for the request (Ex: HTTP/1.1)
$_SERVER['DOCUMENT_ROOT']	Gives file system path to the root folder of the website.
$_SERVER['SCRIPT_NAME']	Path of the current script being executed.
$_SERVER['SERVER_PORT']	Port number the server is using to communicate.

How do you Get Server Information?

We can get Server information using server variables. The code to get server information is given below:

```
<?php
        echo "Server Name: " . $_SERVER['SERVER_NAME'] . "<br>";
        echo "Server Software: " . $_SERVER['SERVER_SOFTWARE'] . "<br>";
        echo "Server Protocol: " . $_SERVER['SERVER_PROTOCOL'] . "<br>";
        echo "Document Root: " . $_SERVER['DOCUMENT_ROOT'] . "<br>";
        echo "Script Name: " . $_SERVER['SCRIPT_NAME'] . "<br>";
        echo "Server Port: " . $_SERVER['SERVER_PORT'] . "<br>";
?>
```

Super Global Variables:

Super global variables in PHP are built-in variables that are always accessible, regardless of scope I.e. inside functions, classes, or files. These variables are used to access data from forms, sessions, server information, cookies, etc.

The list of Super Global Variables is given below:

Super Global Variable	Description
$GLOBALS	Used to access global variables from anywhere in the script. All global variables are stored in this associative array.
$_SERVER	Contains information such as headers, paths, server name, IP address, etc.
$_REQUEST	Used to collect data sent from an HTML form. It can collect data sent using either the GET, POST, or COOKIE method. It is useful when we don't know which method was used in the form.
$_POST	Collects data from HTML form using POST method.
$_GET	Collects data from HTML form using GET method.
$_FILES	Used to handle file uploads from a form. It contains file information like file name, type, size, etc.
$_ENV	Contains environment variables. Environment variables are typically set in the server configuration.
$_COOKIE	Used to retrieve cookie values set on the client side.
$_SESSION	Used to store user data to be used across multiple pages.

$GLOBALS Example:

```
<?php
        $x = 5;
        $y = 10;
        function add()
        {
          $GLOBALS['z'] = $GLOBALS['x'] + $GLOBALS['y'];
        }
        add();
        echo $z; // Output: 15
?>
```

$x and $y are global variables because they are not declared inside any function, class or method.

So, to access these global variable $x and $y inside add() function, we need $GLOBALS super global variable. We can't use $x and $y without $GLOBALS inside functions, classes, etc.

$_SERVER Example:

```
<?php
        echo $_SERVER['SERVER_NAME'];                    // Output: localhost
        echo $_SERVER['REQUEST_METHOD'];                 // Output: GET or POST
?>
```

$_REQUEST Example:

Index.html

```
<form method="post" action="info.php">
      Name: <input type="text" name="name">
      <input type="submit">
</form>
```

Info.php

```
<?php
        echo $_REQUEST['name'];
?>
```

The PHP code will display the name entered in the form. Notice we haven't used $_GET['name'] or $_POST ['name'] in **'info.php'** file. Instead, we have used $_REQUEST method. $_REQUEST collects data Send from both GET and POST.

Example:

If we enter "Bishal Bhat" in the input field generated by index.html file, and submit data, the **Info.php** file will display **Bishal Bhat** as output.

$_POST Example:

Index.html

```
<form method="post" action="info.php">
      Name: <input type="text" name="name">
      <input type="submit">
</form>
```

Info.php

```
<?php
        echo $_POST ['name'];
?>
```

$_POST is used for form data submitted with POST method

Example:

If we enter "Bishal Bhat" in the input field generated by index.html file, and submit data, the **Info.php** file will display **Bishal Bhat** as output.

$_GET Example:

Index.html

```
<form method="get" action="info.php">
      Name: <input type="text" name="name">
      <input type="submit">
</form>
```

Info.php

```
<?php
        echo $_GET ['name'];
?>
```

$_GET is used for form data submitted with GET method
Example:
If we enter “Bishal Bhat” in the input field generated by index.html file, and submit data, the **Info.php** file will display **Bishal Bhat** as output.

$_FILES Example

Index.html

```
<form method="post" action="fileHandle.php" enctype="multipart/form-data">
        <input type="file" name="myfile">
        <input type="submit">
</form>
```

fileHandle.php

```
<?php
                                                        // $_FILES handles file uploads
        echo $_FILES['myfile']['name'];                 // Shows the original filename
        echo $_FILES['myfile']['tmp_name'];             // Shows the temporary path
        echo $_FILES['myfile']['size'];                 // Shows the file size in bytes
        echo $_FILES['myfile']['type'];
                        // Shows the MIME type of the file (e.g., image/png, application/pdf)
?>
```

$_COOKIE Example:

script.php

```
<?php
        // Set cookie
        setcookie("user", "Bishal Bhat", time() + 3600);        // Expires in 1 hour

        // Access cookie value (after reload)
        echo $_COOKIE['user'];                                  // Bishal Bhat
?>
```

$_SESSION Example:

firstScript.php

```
<?php
        session_start();            // Start session

        $_SESSION['username'] = "Bishal Bhat";
?>
```

secondScript.php

```
<?php
        session_start();            // Start session
        echo $_SESSION['username'];
                                    // Output: Bishal Bhat
?>
```

We can store and access Session Data in multiple pages using $_SESSION.

Difference Between:

PHP GET Method	PHP POST Method
➢ Data is sent through the URL.	➢ Data is sent in the request body.
➢ Data is visible to everyone (in the address bar).	➢ Data is hidden from the address bar.
➢ Limited amount of data can be sent.	➢ Large amount of data can be sent.
➢ It cannot upload files.	➢ It can upload files.
➢ It is less secure.	➢ It is more secure.
➢ It can be bookmarked.	➢ It cannot be bookmarked.
➢ It is used for reading or searching data.	➢ It is used for submitting or saving data.
➢ Data is accessed using **$_GET['name'].**	➢ Data is accessed using **$_POST ['name'].**
➢ It is default method if not written in the form tag.	➢ To use POST method, we must write ***method = 'post'*** in the html form.
➢ **Example:** <form method = 'get'> ……. </form>	➢ <form method= 'post'> ……. </form>

Form Processing:

The process of collecting data submitted by a user through an HTML form and then using that data in your PHP script to do something like storing in a database, sending an email, or showing a response is called form processing.

The steps for Form Processing are given below:

1. **Create an HTML Form:** Create a form where the user can enter data.
2. **Submit the Form:** Data is sent using **GET** or **POST** method.
3. **Receive the Data in PHP:** Use **$_GET** or **$_POST** or **$_REQUEST** to collect data.
4. **Process the Data:** Validate, store, display, or use the data as needed.

Example:

form.html

```
<form method="post" action="formProcess.php">
    Name: <input type="text" name="uname">
    <input type="submit">
</form>
```

formProcess.php

```
<?php
    $name = $_POST[uname'];
    echo "Welcome, " . $name;
?>
```

Output:
If user types "Bishal Bhat" in the input field submits the form, then the formProcess.php file will show "Welcome Bishal Bhat".

Note:
If we don't write **method** attribute in the <form> tag. Then, it uses **GET** method by default. So, to access the data sent by this form, we need to use $_GET['InputfieldName'].

Form Attributes:

The syntax of HTML form tag is given as:

```
<form action = "..." method = "..." name = "..." enctype = "..." >
                    .....
</form>
```

1. **Action:**
   It specifies the URL of PHP file where the Form data should be sent for processing. We leave this attribute's value empty (action = "") if we want to handle the form data in the same page.
2. **Method:**
   It specifies how form data should be sent i.e. through GET method or POST method. If we don't specify any value to this attribute, Data is sent using GET method.
3. **Name:**
   Name of the form rarely used.
4. **Enctype:**
   It specifies the **encoding type** used when submitting the form. It is required **only when uploading files**. It value must be set to: **multipart/form-data.**
   If not uploading files, this can be ignored.

Example:

```
<form action="fileProcess.php" method="post" enctype="multipart/form-data">
        <input type="text" name="username">
        <input type="file" name="profilePic">
        <input type="submit">
</form>
```

Form Parameter:

The name – value pair data sent from HTML form to a PHP script is called from parameter. The **name** attribute of the form input field is the **parameter name** and **user's input** is the parameter value.

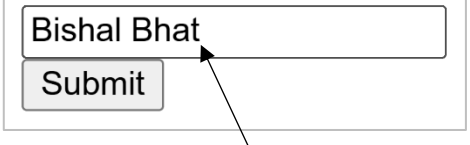
Example:

form.html

```
<form method="post" action="process.php">
        <input type="text" name="username"> <br>
        <input type="submit">
</form>
```

Form parameter name

Form Output



Form parameter Value

File Upload in PHP:

Uploading a file using PHP involves 2 main parts.

1. HTML form to upload file
2. PHP script to handle File upload

Q. Write HTML and PHP code to upload image with size ≤ 2MB.

form.html

```
<form action="upload.php" method="post" enctype="multipart/form-data">
        Upload file:  <br>
         <input type="file" name="myfile">
        <input type="submit" value="submit ">
</form>
```

Upload.php

```
<?php
    if (isset($_FILES['myfile'])) {
        $fileName = $_FILES['myfile']['name'];                  // original name of file
        $tmpName  = $_FILES['myfile']['tmp_name'];              // temp. location of file in server
        $fileSize = $_FILES['myfile']['size'];                  // file size in bytes
        $fileType = $_FILES['myfile']['type'];                  // MIME type of file

                                                                // Allowed extensions
        $allowedTypes = ['image/jpeg', 'image/jpg', 'image/png', 'image/gif'];

                                                                // Check file type and size
        if (!in_array($fileType, $allowedTypes)) {
            echo "Error: Only image files (JPG, JPEG, PNG, GIF) are allowed.";
        } elseif ($fileSize > 2 * 1024 * 1024) { // 2 MB = 2 * 1024 * 1024 bytes
            echo "Error: File must be 2MB or less.";
        } else {
            $uploadPath = "uploads/" . $fileName;
            if (move_uploaded_file($tmpName, $uploadPath)) {
                echo "File uploaded successfully!";
            } else {
                echo "Error: Failed to upload file.";
            }
        }
    }
?>
```

Header Function:

The header() function is used to send raw HTTP headers to the browser before any output. It is mainly used to:

- Redirect to another page
- Set content type (HTML, JSON, etc)
- Manage cache settings
- Set HTTP status code
- Force File downloads

Example:

```php
<?php
        header("Location: welcome.php");                    // Redirects user to "welcome.php"
        exit();                                             // Best practice: stop further script execution
?>
```

Form Validation:

The process of checking if the user input is correct, complete, and safe **before** using or storing it is called form validation. There are 2 types of form validation. They are:

1. Client-Side form validation → It is done using HTML / JavaScript
2. Server-Side Form Validation → It is done using Server-side Scripting language like PHP.

Why server-side Form validation is done even after client-side validation?

Server-side validation is essential even after client-side validation because client-side validation can be bypassed. Malicious users can disable JavaScript or manipulate form data before submission. Server-side validation ensures security, data integrity, and protects against threats like SQL injection and XSS. It acts as a final safeguard, ensuring only valid and safe data is processed by the server.

Some Common Form validations are:

1. Required fields validation (Ensures important fields are not left empty)
2. Numeric Validation (Ensures only numbers are entered)
3. Password Validation (Checks for length, special characters, or strength of password)
4. Length of input (checks for length of input)
5. Email Validation (Verifies input matches a valid email format)
6. File type Validation (Restricts uploads to specific file size & formats)
   // File type validation already done…. Upload image with size ≤ 2MB wala example….

Server-Side validation Examples:

1. **Required Fields validation:**

```php
<?php
        if (empty($_POST['name']))
        {
          echo "Name is required.<br>";
        }
?>
```

2. **Numeric Validation:**

```php
<?php
        $age = $_POST['age'];

        if (! filter_var($age, FILTER_VALIDATE_INT)) {
                echo "Please enter a valid whole number for age.<br>";
        }
?>
```

3. **Password Validation:**

```
<?php
        $password = $_POST['password'];
        if (strlen($password) < 8) {
                echo "Password must be at least 8 characters long.<br>";
        }
        if (! preg_match("/[A-Z]/", $password)) {
                echo "Password must include at least one uppercase letter.<br>";
        }
        if (! preg_match("/[^a-zA-Z\d]/", $password)) {
                 echo "Password must include at least one special character.<br>";
        }
?>
```

4. **Length of Input Validation:**

```
<?php
        $username = $_POST['username'];
        if (strlen($username) < 4 || strlen($username) > 12) {
                 echo "Username must be 4–12 characters long.<br>";
        }
?>
```

5. **Email Validation:**

```
<?php
        $email = $_POST['email'];
        if (! filter_var($email, FILTER_VALIDATE_EMAIL)) {
                echo "Invalid email format.<br>";
        }
?>
```

Note:

filter_var() is a built-in PHP function used to filter and validate data (like emails, numbers, URLs, etc.) The syntax is:

filter_var (variable, filter_type)

- varibale → The data we want to check or validate
- filter-type → A filter type in PHP is a predefined constant used with the filter_var() function to specify what kind of check we want to perform on a variable.

Some filter_types are given as:

- FILTER_VALIDATE_INT → Checks if value is a valid Integer
- FILTER_VALIDATE_FLOAT → Checks if value is valid floating number.
- FILTER_VALIDATE_EMAIL → Checks if value has valid email format.

COOKIE:

A cookie is a small piece of data stored on the client's browser. It is used to store information that can be accessed across different pages. Cookies are commonly used for tasks such as user preferences, session management, or tracking.

Setting a Cookie: *(Creating a cookie)*

```
setcookie(name, value, expire, path);
```

- Name → Name of cookie
- Value → Value of cookie
- Expire → Expiration time of the cookie in seconds. If not set, the cookie will expire at the end of session. We use time() function get current time in seconds.
- Path → Path of server where cookie will be available.

```
setcookie("user", "Bishal Bhat",  time() + 3600, "/");          // This cookie expires in 1 Hour
```

Update a Cookie:

To update a cookie, simply set it with same name but different value or expiration time.

```
setcookie("username", "Simran", time() + 7600, "/");                    // Updates the value
```

Read a Cookie:

To read a cookie, we use $_COOKIE super global variable.

```
if (isset($_COOKIE["username"])) {
        echo "Hello, " . $_COOKIE["username"];                    // Hello, Simran
} else {
        echo "Cookie is not set.";
}
```

Delete a Cookie:

To delete a cookie, we simply set its expiration to a past time.

```
setcookie("username", "Simran", time() - 100, "/");                    // Deletes the cookie
```

Session:

Session is a way to store user information to be used across multiple pages. Unlike cookies, which stores data on the user's browsers, session data is stored on the server, which makes it more secure. We should always start session at the top of our code. Session is used for:

- Maintaining user login states.
- Preserving user preferences across pages.
- Tracking user activity during a visit.
- Storing shopping cart contents.

A session ID is a randomly generated string that uniquely identifies a user's session on the server. By default, a PHP session lasts until the user closes the browser, manually ends the session, or it expires based on the server's configuration—typically after 24 minutes of inactivity.

Session Creation:
To create or start a session in PHP, use the **session_start()** function. This function should be called at the beginning of the script, before any HTML output.

```
<?php
        session_start();                                                    // Start the session
?>
```

Adding Data to a Session:
Once a session is started, you can store data in the session using $_SESSION super global variable.

```
<?php
session_start();                                                    // Start the session
                                                                    // Add data to the session
$_SESSION['username'] = Bishal Bhat';                               // Storing username
$_SESSION['email'] = 'bb@gmail.com';                                // Storing email
$_SESSION['age'] = 23;                                              // Storing age

                                        // This session data is now available throughout the session
?>
```

Reading Session Data:
After data is stored in the session, we can access it on any page where the session is active.

```
<?php
        session_start();                                            // Start the session
                                                                    // Access session data
        echo $_SESSION['username'];                                 // Outputs: Bishal Bhat
        echo $_SESSION['email'];                                    // Outputs: bb@gmail.com
        echo $_SESSION['age'];                                      // Outputs: 23
?>
```

Updating Session Data:
To update data in the session, simply reassign the value of the specific session variable.

```
<?php
        session_start();                                            // Start the session
        $_SESSION['username'] = 'Simran';                           // Change the username
        echo $ _SESSION['username'];                                // updated Output: Simran
?>
```

Deleting Specific Session Data:
If we want to remove a particular session variable, we can use unset().

```
<?php
        session_start();                                            // Start the session
        unset($_SESSION['username']);                               // Removes 'username' from the session
                                                // Now, $_SESSION['username'] will be undefined
?>
```

Destroying the Entire Session:
To completely destroy a session and remove all session data, we use **session_unset()** and **session_destroy().**

```
<?php
        session_start();                                    // Start the session
        session_unset();                                    // Clears all session data
        session_destroy();                                  // Destroys the session
                                // Now, all session data is cleared, and the session is destroyed
?>
```

Difference Between:

Cookie	Session
➢ Data is stored on the user's browser.	➢ Data is stored on the server.
➢ Can store small amounts of data (up to 4KB).	➢ Can store larger amounts of data.
➢ Data persists even after the browser closed (until cookie expires).	➢ Data disappears when the browser is closed.
➢ Useful for remembering user preferences.	➢ Useful for storing user-specific data during a session.
➢ Can be manipulated by the user.	➢ More secure; cannot be easily manipulated by the user.
➢ Set with **setcookie()** function.	➢ Started with **session_start()** function and value are set using **$_SESSION** super global variable.

Secure Sockets Layer (SSL):
SSL (Secure Sockets Layer) is a security technology used to create a secure, encrypted connection between a web browser and a server. It helps protect sensitive data like passwords and personal information from hackers. Website using SSL start with "https://" instead of "http://" and shows a padlock icon in the browser. It builds user trust, improves search rankings, and is important for safe online communication.

Session Example for Login:
When the user is logged in, he/she can access the website. But when the user is logged out, the website must be inaccessible to him/her. We can do this using session. The simple example using session to protect website from unauthorized access is given below:

Here, we made 3 files → login.php, dashboard.php, and logout.php.
When the user is logged in, he can access dashboard.php and cannot access loginpage.
When the user is logged out, he cannot access dashboard.php and can only access login.php page.

login.php

```php
<?php
    session_start();                                    # Session always start at the top
    $correct_username = "bishalbhat2002";
    $correct_password = "0000";
    if(isset($_POST['submit'])){                        # Check if the form is submitted
        if(!empty($_POST['username']) && !empty($_POST['password'])){
            # Get the username and password from the form
            $username = $_POST['username'];
            $password = $_POST['password'];
            # Check if username and password match
            if ($username == $correct_username && $password == $correct_password) {
                $_SESSION['username'] = $username;              # set session variables
                header("Location: dashboard.php");          # Redirect to dashboard.php
                exit();
            } else {
                $error_message = "Wrong username or password!";
            }
        }
    }
    if (isset($_SESSION['username'])) {         # If user is already logged in, redirect to dashboard
    header("Location: dashboard.php");
    exit();
    }
?>

<html>
<head>
  <title>Login</title>
</head>
<body>
  <h2>Login Page</h2>

  <?php
    if (isset($error_message)) {                        # Display error message if login failed
        echo "<p style='color:red;'>$error_message</p>";
    }
  ?>

  <form action="" method="POST">
        Username: <input type="text" name="username" required><br><br>
        Password: <input type="password" name="password" required><br><br>
        <input type="submit" name="submit" value="submit">
  </form>
</body>
</html>
```

dashboard.php

```
<?php
    session_start();
    if (!isset($_SESSION['username'])) {                    # Check if user is logged in
            header("Location: login.php");                  # Redirect to login page if not logged in
            exit();
    }
?>

<html>
<head>
  <title>Dashboard</title>
</head>
<body>
        <h2>Welcome to your Dashboard!</h2>
        <p>You are logged in as <?php echo $_SESSION['username']; ?>.</p>
        <a href="logout.php">Logout</a>
</body>
</html>
```

logout.php

```
<?php
    session_start();
    session_unset();                                        # Unset Session Variables
    session_destroy();                                      # Destroys Session
    header("Location: login.php");                          # Redirect to login page
    exit();
?>
```

***

Unit 5

Database Connectivity

RDBMS:

RDBMS stands for Relational Database Management System. It is a software used to store and manage data in tables (called relations) in the database. In RDBMS, data is organized into rows and columns, and tables can be linked to each other using keys (like primary key and foreign key). It uses SQL for performing database operations. Examples of RDBMS are MySQL, Oracle, PostgreSQL, etc. RDBMS ensures data integrity, accuracy, and easy access to related data.

MySQL:

MySQL is a free and open-source RDBMS. It is used to store, manage, and organize the data. It uses SQL (Structured Query Language) to perform tasks like adding, updating, deleting, and retrieving data from databases. MySQL is commonly used with PHP in web development to build dynamic websites. MySQL supports features like tables, rows, primary keys, and foreign keys, and it is fast, reliable, and works well with large databases. It is widely used in CMS, e-commerce, and many online applications.

Connect PHP with MySQL Server:

`We can connect PHP with MySQL in 3 ways. They are:

1. MySQLI (Procedural)
2. MySQLI (object Oriented)
3. PDO

MySQL Server Connection using MySQLI object-oriented approach:

PHP code to Connect MySQL Server

```
<?php
        $servername = "localhost";
        $username = "root";
        $password = "";

        try {
                                                    // Create connection using object-oriented approach
                $conn = new mysqli ( $servername, $username, $password );
                echo "MySQL Server Connected successfully";
        }
        catch (Exception $e) {
                die ( "Error While connecting: ".$e->getMessage() );        // Display the error message
        }
?>
```

Note:

In PHP 8 and later, mysqli throws an exception if there's an issue with the database connection or query execution. To handle these errors effectively, always use a try-catch block when connecting to the database or executing SQL queries. This ensures proper error handling and prevents unexpected crashes.

Database Connection using MySQLI Object-oriented approach:

PHP code to Connect Database [db_connect.php]

```
<?php
        $servername = "localhost";
        $username = "root";
        $password = "";
        $dbname = "myDatabase";

        try {
                                                // Create connection using object-oriented approach
                $conn = new mysqli ( $servername, $username, $password, $dbname );
                echo " Database Connected successfully";
        }
        catch (Exception $e) {
                die ( "Error While connecting: ".$e->getMessage() );        // Display the error message
        }
?>
```

SQLite:

SQLite is a lightweight, serverless, and self-contained database engine that stores the entire database in a single file. It requires no setup or server installation, making it ideal for small to medium applications. SQLite is cross-platform, easy to use, and works well for local or embedded systems like mobile apps and desktop tools. It supports standard SQL queries for managing data efficiently.

Data Manipulation Language (DML):

DML stands for Data Manipulation Language. It includes SQL commands used to manipulate data stored in the database. These operations do not define or change the database structure but instead work with the actual data. The DML commands are:

1. **INSERT:** Adds a new record to a table.
   INSERT INTO users (name, email, age) VALUES ('Bishal Bhat', 'bb@gmil.com', 23);
2. **SELECT:** Retrieves data from a table.
   SELECT * FROM users Where name = "Bishal Bhat";
3. **UPDATE:** Modifies existing records.
   UPDATE users SET name = 'Simran' WHERE id = 1;
4. **DELETE:** Removes record from a table.
   DELETE FROM users WHERE id = 1;

Note:

In PHP, SQL queries are typically written as strings and stored in a variable (e.g., $sql). These queries are then executed using $conn → query($sql) method when using **MySQLi** object-oriented approach. This approach is used for executing queries like CREATE, SELECT, INSERT, UPDATE, DELETE, JOIN, etc.

Creating Database Using PHP:

Db_config.php

```
<?php
        $server = "localhost";
        $username = "root";
        $password = "";
        $dbname = "myDatabase";                                    # Database Name

        # Connecting with MYSQL Server...
        try {
                $conn = new mysqli($server, $username, $password);      # Creates connection obj.
        } catch (Exception $e) {
                die("<b>Error While connecting MySQL: </b>" . $e->getMessage());
        }

        # Creating database if not already Exists...
        try {
                $sql = "CREATE database if not exists $dbname";
                $conn->query($sql);                                # SQL Query Executed here ...
                $conn->select_db($dbname);                 # Selecting database after creation...
        } catch (Exception $e) {
                die("<b>Error While Creating Database: </b>" . $e->getMessage());
        }

        try {
                 $sql = "CREATE table if not EXISTS users (
                            id Int Auto_increment PRIMARY KEY,
                            username varchar(50) Not Null unique,
                            password varchar(100) not null
                          )";
                $conn->query($sql);                                # SQL Query Executed here ...
        } catch (Exception $e) {
                die("<b>Error While Creating Table : </b>" . $e->getMessage());
        }
?>
```

This file first set connection with MySQL server using Object oriented approach. And then Creates “”myDatabase” database if not already exists, then selects the created database.

After that creates “users” table if not already exists.
If specified database and tables already exists, then it will not create new database and tables.

If any errors occurred during connection or creation, respective catch block will handle the error.

Database and Table Created:

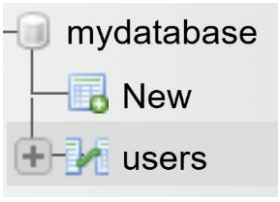


Table Structure:

id	username	password

Inserting Data into the Database:

Insert_data.php

```
<html>
    <head>
        <title> INSERT EXAMPLE </title>
    </head>
    <body>
        <form action="" method="post">
            Username: <input type="text" name="uname"> <br>
            Password: <input type="text" name="password"> <br>
            <input type="submit" name="submit">
        </form>
    </body>
</html>

<?php
        # Include db_connect.php file code to connect to database...
        require_once "db_connect.php";

        # Check if submit button was pressed or not ...
        if (isset($_POST['submit'])) {
            if (!empty($_POST['uname']) && !empty($_POST['password'])) {

                # Store all the data in variables send by FORM through POST method
                $uname = $_POST['uname'];
                $password = $_POST['password'];

                # Insert into Users table
                try {
                    $sql = "INSERT INTO USERS (username, password) values
                            ('$uname', '$password')";
                    $conn->query($sql);                    # SQL Query Executed here ...
                    echo "<br>Data inserted successfully...";

                } catch (Exception $e) {
                    die("<b>Error While Inserting data: </b>" . $e->getMessage());
                }
            } else {
                    echo "<b>User Must fill all form fields...</b>";
            }
        }
?>
```

The line require_once "db_connect.php"; we are including database connection code from “db_connect.php” file. The code for database connection is written in 2nd page of this unit. We have reused that code here.....

Exam ma yesto question aaye, first ma yo unit ko second page “db_connect.php” file wala code alag lekhne ani tyo code lai “db_connect.php” name dine. And feri yo code lekhne.... SELECT, UPDATE, and DELETE ma ni testai garne ho ....

Output FORM:

Username: Bishal Bhat
Password: I-love-coding
Submit

Data inserted in Users table:

id	username	password
1	Bishal Bhat	I-love-coding

Viewing Data from the Database:

view_data.php

```
<html>
    <head>
        <title> SELECT EXAMPLE </title>
    </head>
    <body>

        <?php
            require_once "db_connect.php";
            try {
                    $sql = "SELECT id, username, password from users";
                    $result = $conn->query($sql);
            } catch (Exception $e) {
                    die("<b>Error While Fetching Users: </b>" . $e->getMessage());
            }
            if ($result->num_rows > 0) {
        ?>
                <table border="1">
                    <tr>
                        <td>id</td>
                        <td>username</td>
                        <td>password</td>
                    </tr>
                    <?php
                        while ($row = $result->fetch_assoc()) {
                     ?>
                            <tr>
                                    <td><?php echo $row['id'];  ?></td>
                                    <td><?php echo $row['username'];  ?></td>
                                    <td><?php echo $row['password'];  ?></td>
                            </tr>
                    <?php
                        }
                    ?>
                </table>
        <?php
            }else{
                    echo "No Recound Found !!";
            }
        ?>
    </body>
</html>
```

Data in the Database:

id	username	password
1	Bishal Bhat	I-love-coding
2	Simran	12345
3	Jethalal	00000

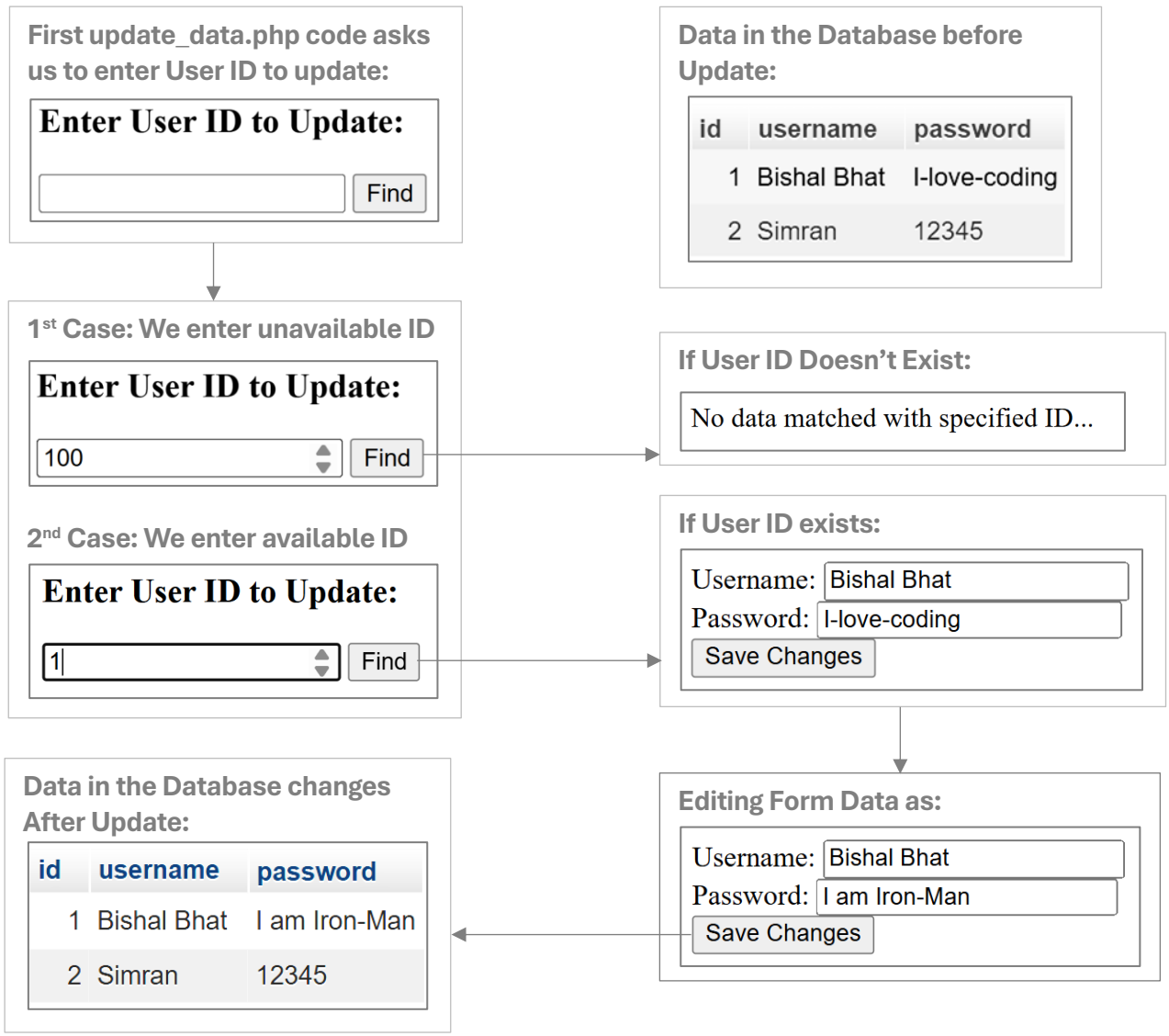
View_data.php file output:

id	username	password
1	Bishal Bhat	I-love-coding
2	Simran	12345
3	Jethalal	00000

Update Data in the Database:

```
<html>
    <head>
        <title> UPDATE EXAMPLE </title>
    </head>
    <body>
<?php
        require_once 'db_connect.php';
        if (isset($_GET['id'])) {
            $id = $_GET['id'];                          # GET update id from URL and store in variable...
            try {
                $sql = "SELECT * from Users
                        WHERE id = $id";                # Select Data Which we want to update...
                $result = $conn -> query($sql);
            } catch (Exception $e) {
                die("<b>Error: </b>" . $e->getMessage());
            }

            if ($result->num_rows > 0) {
                $row = $result->fetch_assoc();
?>
                 <form action="" method="post">         # Show Data Selected data inside Form to update...
                        <input type="hidden" name="id" value="<?php echo $row['id']; ?>">
                        Username:
                        <input type="text" name="uname" value="<?php echo $row['username']; ?>">
                        <br>
                        Password:
                        <input type="text" name="password" value="<?php echo $row['password']; ?>">
                        <br>

                        <input type="submit" name="submit" value="Save Changes " >
             </form>
<?php
                } else {
                        echo "No data matched with specified ID...";
                }
        } else {
                echo "<h3>Enter User ID to Update:</h3>";
                echo "<form action='' method='GET'>                  # Form to select ID to update user
                        <input type='number' name='id' min='1' step='1' >
                        <input type='submit' value='Find'>
                    </form>";
        }
?>
                                                                    # Update Process code in the same file ...
<?php
          if (isset($_POST['submit'])) {
                if (!empty($_POST['uname']) && !empty($_POST['password'])) {
                        $id = $_POST['id'];                         # store updated data in variables
                        $uname = $_POST['uname'];
```

```
                $password = $_POST['password'];

                try {
                    $sql = "UPDATE Users
                            SET username = '$uname', password = '$password'
                            where id = $id";
                    $conn->query($sql);                    # Update query executed here
                    header('location: data_view.php');
                                                # Redirects to Data_view.php page to view changes...
                } catch (Exception $e) {
                    die("<b>Error While Updating Data: </b>" . $e->getMessage());
                }
            } else {
                echo "User must fill all the input fields...";
        }
?>

    <body>
<html>
```

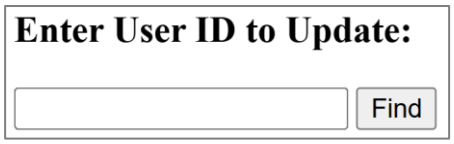


Delete Data from Database:

```
<html>
    <head>
        <title> DELETE EXAMPLE </title>
    </head>
    <body>
        <?php
            require_once 'db_connect.php';                  # Connect to database
            if(isset($_GET['id'])){                         # If user Id is provided in URL for deletion
                $id = $_GET['id'];

                try{
                    $sql = "Delete from users where id = $id";
                    $conn->query($sql);
                    header('location: data_view.php');
                                                    # Redirects to data_view.php page to View Deletion
                }catch(Exception $e){
                    die("<br><b>Error while deleting Record:</b>".$e->getMessage());
                }
            }else{
                echo "<h3>Enter User ID to Delete:</h3>";
                echo "<form action='' method='GET'>
                            <input type='number' name='userid' min='1' step='1' >
                            <input type='submit' value='Find'>
                        </form>";

                if(isset($_GET['userid'])) {                # Check if userid is given in URL
                    $userid = $_GET['userid'];              # Copy userid in variable from URL

                    try{
                            $sql = "SELECT * from Users
                                    WHERE id = $userid";
                            $result = $conn->query($sql);
                    }catch(Exception $e){
                            die("<b>Error: </b>".$e->getMessage());
                    }

                    if($result->num_rows>0){            # Display Info about user if found in database
                        $row = $result->fetch_assoc();
                    ?>
                    <form action="" method="GET">
                        <input type="hidden" name="id" value="<?php echo $row['id']; ?>">
                        Username:
                        <input type="text" value="<?php echo $row['username']; ?>" readonly> <br>
                        Password:
                        <input type="text" value="<?php echo $row['password']; ?>" readonly> <br>
                        <input type="submit" value="Delete" name="submit">
                    </form>
                <?php
                    }else{
```

```
                    echo "No User found with Specified ID...";
                }
            }
        }
    ?>
  </body>
</html>
```

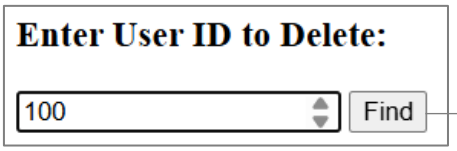
First delete_data.php code asks us to enter User ID to delete:



Data in the Database before Update:

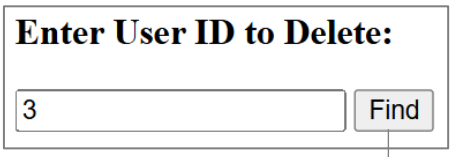
id	username	password
1	Bishal Bhat	I-love-coding
2	Simran	12345
3	Jethalal	00000

1st Case: We enter unavailable ID

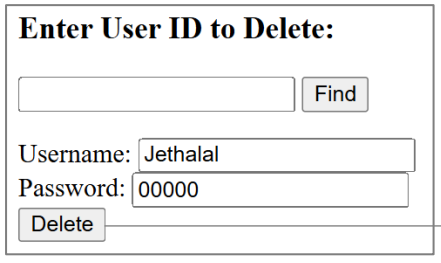


If User ID Doesn't Exist:

2nd Case: We enter available ID



If User ID exists:



On clicking Delete button, the page is redirect to view_data.php page to view Deletion:

id	username	password
1	Bishal Bhat	I-love-coding
2	Simran	12345

Data in the Database After Update:

id	username	password
1	Bishal Bhat	I-love-coding
2	Simran	12345
3	Jethalal	00000

SQL Injection:

SQL Injection is a hacking technique where the hacker inserts malicious SQL code through user input fields to access, modify, or destroy data in the database.

Example of Vulnerable Code with Username and Password:

```
$username = $_POST['username'];
$password = $_POST['password'];
$sql = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
```

If hacker enters following strings in the input fields, then the hacker can easily access the website.

Username: ' OR '1'='1
Password: ' OR '1'='1

With above values SQL statement becomes:
SELECT * FROM users WHERE **username = '' OR '1'='1' AND password = '' OR '1'='1';**

Where condition is always true in this condition, and any unauthorized user can access the website. This returns all users. A major security risk. So, we must prevent this type of attacks.

How to Prevent SQL Injection?

SQL injection can be prevented by using Prepared statements.

Prepared Statements:

Prepared statements separate SQL logic from user input using **placeholders (?)**, then safely **bind** user data to those placeholders. The benefits of Prepared Statements are given below:

- Prevent SQL injection
- Escape special characters automatically
- Reuse query for multiple values
- Cleaner and readable code
- Safe input handling by database

The syntax of Prepared Statement is given below:

```
$stmt = $conn → prepare ("SQL query with ? placeholders");
$stmt → bind_param ("data type", $value);                    // "s" = string
$stmt → execute();
$result = $stmt → get_result ();                             // for SELECT only
```

- prepare() → Prepares sql query with **?** placeholders.
- bind_param() → Binds variables to the placeholders in the prepared SQL statement, specifying their data types. The values must be provided in the exact order in which the placeholders appear in the query.
- execute() → Executes the prepared statement.
- get_result() → Fetches result set (only used for SELECT statement. For INSERT, UPDATE, DELETE we don't use the 4th statement).
- feth_assoc() → Gets one row from $result as associative array.

Data Types used in bind_param():

- i → integer
- s → string
- d → double (float)
- b → blob

CRUD EXAMPLES using Prepared Statements:

1. **SELECT EXAMPLE:**

```
$sql = "SELECT * FROM users WHERE username = ?";
$stmt = $conn → prepare ($sql);
$stmt → bind_param("s", $uname);
$stmt → execute();
$result = $stmt → get_result();

while ($row = $result->fetch_assoc()) {
        echo $row["username"];
}
```

2. **INSERT EXAMPLE:**

```
$sql = "INSERT INTO users (username, password) VALUES (?, ?)";
$stmt = $conn → prepare($sql);
$stmt → bind_param("ss", $uname, $password);
$stmt → execute();
```

3. **UPDATE EXMAPLE:**

```
$sql = "UPDATE users SET password = ? WHERE username = ?";
$stmt = $conn → prepare($sql);
$stmt → bind_param("ss", $newPass, $uname);
$stmt → execute();
```

4. **DELETE EXAMPLE:**

```
$sql = "DELETE FROM users WHERE username = ?";
$stmt = $conn → prepare($sql);
$stmt → bind_param("s", $uname);
$stmt → execute();
```

In Simple words, prepare() function bhitra hami incomplete sql query lekhxau. incomplete matlab, user input bina ko query, user le input gareko value hunu parne thau ma hami placeholders (?) use garxau. Yesko matlab '?' bhayeko thauma hami paxi value halne hau ho...
Bind_param() function le tyo '**?**' ko thau ma value haru lai halne kam garxa. And execute() function le prepared statement lai execute garne kaam garxa.. Prepare() function bhitra lekeheko function lai prepared statement bhaninxa..

$stmt is prepared statement object – bind_function(), execute() function haru lai access garna use garxau.

SQL TABLE CREATE AND INSERT DUMMY DATA FOR JOINS:

To study joins, we need tables. So, the SQL code to create tables are given below:

```
-- Create customers table
    CREATE TABLE customers (
        cid INT PRIMARY KEY AUTO_INCREMENT,
        name VARCHAR(50) NOT NULL,
        email VARCHAR(100) UNIQUE,
        city VARCHAR(50)
    );

-- Create orders table
    CREATE TABLE orders (
        oid INT PRIMARY KEY AUTO_INCREMENT,
        order_date DATE NOT NULL,
        amount DECIMAL(10, 2) NOT NULL,
        cid INT,
        FOREIGN KEY (cid) REFERENCES customers(cid)
    );
```

Now, we Insert data into these tables as:

```
-- Customers table
INSERT INTO customers (name, email, city)
VALUES
('Bishal',  'bishal@gmail.com',  'Mnr'),
('Simran',  'simran@gmail.com',  'Pokhara'),
('Babita',  'babita@gmail.com',  'Lumbini'),
('Jethalal', 'jethalal@gmail.com', 'Kathmandu'),
('Akshay',  'akshay@gmail.com',  'Butwal'),
('Salman',  'salman@gmail.com',  'Birgunj'),
('Varun',   'varun@gmail.com',   'Nepalgunj');


-- Orders table
INSERT INTO orders (order_date, amount,  cid)
VALUES
('2024-01-05', 500.00, 1),
('2024-01-15', 750.00, 2),
('2024-02-10', 250.00, 1),
('2024-03-12', 300.00, 3),
('2024-03-18', 450.00, 5),
('2024-04-01', 900.00, 6),
('2024-04-12', 600.00, 2);
```

cid	name	email	city
1	Bishal	bishal@gmail.com	Mnr
2	Simran	simran@gmail.com	Pokhara
3	Babita	babita@gmail.com	Lumbini
4	Jethalal	jethalal@gmail.com	Kathmandu
5	Akshay	akshay@gmail.com	Butwal
6	Salman	salman@gmail.com	Birgunj
7	Varun	varun@gmail.com	Nepalgunj

oid	order_date	amount	cid
1	2024-01-05	500.00	1
2	2024-01-15	750.00	2
3	2024-02-10	250.00	1
4	2024-03-12	300.00	3
5	2024-03-18	450.00	5
6	2024-04-01	900.00	6
7	2024-04-12	600.00	2

SQL JOINS:

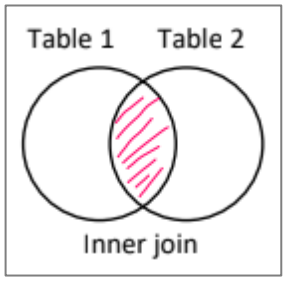
JOIN is used to combine rows from two or more tables based on a related column between them (usually a foreign key). The types of Joins are:

1. Inner join
2. Natural join
3. Left Join (Left outer Join)
4. Right Join (Right outer Join)
5. Full join (Full outer Join)
6. Cross Join

INNER JOIN:

Inner join is the simplest and most common type of join. It is also known as Simple join. 'INNER JOIN" clause is used to perform inner join between the tables and **ON** keyword is used to specify the join condition. It doesn't eliminate duplicate column's from the resultant table.

```
SELECT *
FROM customers
INNER JOIN orders
ON customers.cid = orders.cid;
```



Output:
Only customers who have placed atleast one order.

cid	name	email	city	oid	order_date	amount	cid
1	Bishal	bishal@gmail.com	Mnr	1	2024-01-05	500.00	1
2	Simran	simran@gmail.com	Pokhara	2	2024-01-15	750.00	2
1	Bishal	bishal@gmail.com	Mnr	3	2024-02-10	250.00	1
3	Babita	babita@gmail.com	Lumbini	4	2024-03-12	300.00	3
5	Akshay	akshay@gmail.com	Butwal	5	2024-03-18	450.00	5
6	Salman	salman@gmail.com	Birgunj	6	2024-04-01	900.00	6
2	Simran	simran@gmail.com	Pokhara	7	2024-04-12	600.00	2

NATURAL JOIN:

Automatically joins tables using **columns with the same name** and compatible types. No need to specify condition. Therefore, name and domain of common column must be same for both the tables. We don't need to specify the Join condition using **ON** Keyword here. It eliminates duplicate columns form the resultant table, keeping only one column for each common column's. NATURAL JOIN clause is used to perform natural join. We should avoid use of NATURAL JOIN if columns names and datatypes are not same.

```
SELECT *
FROM customers NATURAL JOIN orders;
```

Output: Output is same as INNER JOIN, only difference is that the resultant table has only distinct columns. INNER JOIN table has 2 cid columns where as NATURAL JOIN has 1 cid column.

cid	name	email	city	oid	order_date	amount
1	Bishal	bishal@gmail.com	Mnr	1	2024-01-05	500.00
2	Simran	simran@gmail.com	Pokhara	2	2024-01-15	750.00
1	Bishal	bishal@gmail.com	Mnr	3	2024-02-10	250.00
3	Babita	babita@gmail.com	Lumbini	4	2024-03-12	300.00
5	Akshay	akshay@gmail.com	Butwal	5	2024-03-18	450.00
6	Salman	salman@gmail.com	Birgunj	6	2024-04-01	900.00
2	Simran	simran@gmail.com	Pokhara	7	2024-04-12	600.00

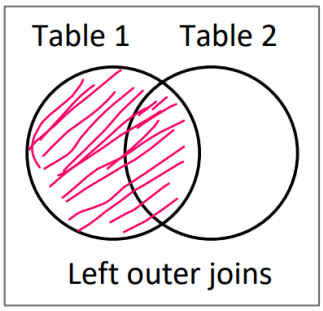
LEFT JOIN (LEFT OUTER JOIN):

The join operation in which the resultant table contains matched as well as unmatched rows of the participating tables is called Outer Join.

LEFT JOIN returns **all records from the left table (customers)** and matched records from the right (orders). The unmatched rows of second table are set to NULL in the resultant table. "LEFT OUTER JOIN" or ""LEFT JOIN" clause is used to perform LEFT JOIN. **ON** keyword is used to specify join condition.

```
SELECT *
FROM customers LEFT JOIN orders
ON customers.cid = orders.cid;
```

Output:
All customers, including those who did not place any order (like Jethalal and Varun).



cid	name	email	city	oid	order_date	amount	cid
1	Bishal	bishal@gmail.com	Mnr	1	2024-01-05	500.00	1
1	Bishal	bishal@gmail.com	Mnr	3	2024-02-10	250.00	1
2	Simran	simran@gmail.com	Pokhara	2	2024-01-15	750.00	2
2	Simran	simran@gmail.com	Pokhara	7	2024-04-12	600.00	2
3	Babita	babita@gmail.com	Lumbini	4	2024-03-12	300.00	3
4	Jethalal	jethalal@gmail.com	Kathmandu	*NULL*	*NULL*	*NULL*	*NULL*
5	Akshay	akshay@gmail.com	Butwal	5	2024-03-18	450.00	5
6	Salman	salman@gmail.com	Birgunj	6	2024-04-01	900.00	6
7	Varun	varun@gmail.com	Nepalgunj	*NULL*	*NULL*	*NULL*	*NULL*

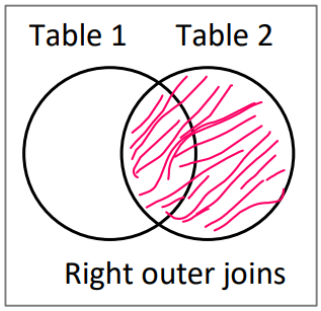
RIGHT JOIN (RIGHT OUTER JOIN):

RIGHT JOIN returns **all records from the right table (orders)** and matched records from the right (customers). The unmatched rows of first table are set to NULL in the resultant table. RIGHT JOIN or

RIGHT OUTER JOIN clause is used to perform RIGHT JOIN. **ON** keyword is used to specify join condition.

```
SELECT *
FROM orders RIGHT JOIN customers
ON orders.cid = customers.cid;
```

Output:
All customers, including those who did not place any order (like Jethalal and Varun).

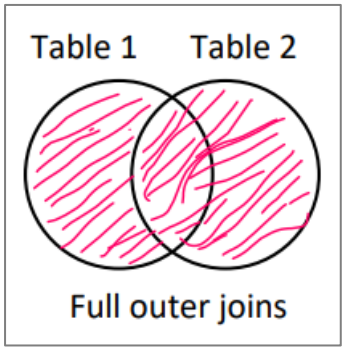


oid	order_date	amount	cid	cid	name	email	city
1	2024-01-05	500.00	1	1	Bishal	bishal@gmail.com	Mnr
3	2024-02-10	250.00	1	1	Bishal	bishal@gmail.com	Mnr
2	2024-01-15	750.00	2	2	Simran	simran@gmail.com	Pokhara
7	2024-04-12	600.00	2	2	Simran	simran@gmail.com	Pokhara
4	2024-03-12	300.00	3	3	Babita	babita@gmail.com	Lumbini
NULL	*NULL*	*NULL*	*NULL*	4	Jethalal	jethalal@gmail.com	Kathmandu
5	2024-03-18	450.00	5	5	Akshay	akshay@gmail.com	Butwal
6	2024-04-01	900.00	6	6	Salman	salman@gmail.com	Birgunj
NULL	*NULL*	*NULL*	*NULL*	7	Varun	varun@gmail.com	Nepalgunj

FULL JOIN (FULL OUTER JOIN):
Full outer join returns rows from both the tables. When no matching rows exist for left table in right table, then the columns of right table are set to NULL and vice versa. No MySQL support FULL OUTER JOIN directly. But we can simulate full outer join using left outer join and right outer join and combining the result using UNION keyword.

```
SELECT *
FROM customers LEFT JOIN orders
ON customers.cid = orders.cid

UNION

SELECT *
FROM customers RIGHT JOIN orders
ON customers.cid = orders.cid;
```

Output:
All customers and all orders, matched where possible, otherwise NULLs.



cid	name	email	city	oid	order_date	amount	cid
1	Bishal	bishal@gmail.com	Mnr	1	2024-01-05	500.00	1
1	Bishal	bishal@gmail.com	Mnr	3	2024-02-10	250.00	1
2	Simran	simran@gmail.com	Pokhara	2	2024-01-15	750.00	2
2	Simran	simran@gmail.com	Pokhara	7	2024-04-12	600.00	2
3	Babita	babita@gmail.com	Lumbini	4	2024-03-12	300.00	3
4	Jethalal	jethalal@gmail.com	Kathmandu	*NULL*	*NULL*	*NULL*	*NULL*
5	Akshay	akshay@gmail.com	Butwal	5	2024-03-18	450.00	5
6	Salman	salman@gmail.com	Birgunj	6	2024-04-01	900.00	6
7	Varun	varun@gmail.com	Nepalgunj	*NULL*	*NULL*	*NULL*	*NULL*

SELF JOIN:

A Self Join is a type of join where a table is joined with itself. A self-join is useful when you want to **compare rows within the same table**, especially when rows represent entities that relate to each other. Some common scenarios where SELF JOIN is used are:

- An employee reports to another employee in the same table.
- A product is compared with another product from the same table.
- You want to find pairs of similar data within the same dataset.

To join a table with itself, we give **aliases** to the table (like A and B) so SQL can treat them as two different tables. We use **JOIN** keyword to perform SELF JOIN and ON keyword to specify join condition. The syntax of SELF JOIN IS:

```
SELECT A.column1, B.column2
FROM table_name A
JOIN table_name B
ON A.common_column = B.common_column;
```

CROSS JOIN (CARTESIAN PRODUCT JOIN):

The "CROSS JOIN" clause is used to perform cross join between any 2 tables. The "cross join" joins each row form one table with all rows from another table. I.e. If we have x rows in table1 and y rows in table2 and performed CROSS JOIN between table1 and table2 then resultant table has x*y rows. It gives cartesian product of rows. Hence, it is called cartesian product join. In Cross join, no joining conditions are specified. And when conditions are specified in cross join then, cross join acts as inner join.

```
SELECT *
FROM customers CROSS JOIN orders;
```

Output:
All customers and all orders, matched where possible, otherwise NULLs.

cid	name	email	city	oid	order_date	amount	cid
1	Bishal	bishal@gmail.com	Mnr	1	2024-01-05	500.00	1
2	Simran	simran@gmail.com	Pokhara	1	2024-01-05	500.00	1
3	Babita	babita@gmail.com	Lumbini	1	2024-01-05	500.00	1
4	Jethalal	jethalal@gmail.com	Kathmandu	1	2024-01-05	500.00	1
5	Akshay	akshay@gmail.com	Butwal	1	2024-01-05	500.00	1
6	Salman	salman@gmail.com	Birgunj	1	2024-01-05	500.00	1
7	Varun	varun@gmail.com	Nepalgunj	1	2024-01-05	500.00	1
1	Bishal	bishal@gmail.com	Mnr	2	2024-01-15	750.00	2
2	Simran	simran@gmail.com	Pokhara	2	2024-01-15	750.00	2
3	Babita	babita@gmail.com	Lumbini	2	2024-01-15	750.00	2
4	Jethalal	jethalal@gmail.com	Kathmandu	2	2024-01-15	750.00	2
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
4	Jethalal	jethalal@gmail.com	Kathmandu	7	2024-04-12	600.00	2
5	Akshay	akshay@gmail.com	Butwal	7	2024-04-12	600.00	2
6	Salman	salman@gmail.com	Birgunj	7	2024-04-12	600.00	2
7	Varun	varun@gmail.com	Nepalgunj	7	2024-04-12	600.00	2

Thau bachauna lai bich ma dot (....) use gareko ho... yesko matlab yo thau ma aru rows haru xan ho .... Yo table ma total 7 x 7 = 49 rows xan...

Note:
For simplicity, I have selected all columns (*) in all above join examples. We can select particular columns from participating tables as well using syntax → *tableName.columnName*
Example:

```
SELECT customers.name, orders.order_date, orders.amount
FROM customers
INNER JOIN orders
ON customers.cid = orders.cid;
```

Output: Only 3 columns values are shown in the resultant table.

name	order_date	amount
Bishal	2024-01-05	500.00
Simran	2024-01-15	750.00
Bishal	2024-02-10	250.00
Babita	2024-03-12	300.00
Akshay	2024-03-18	450.00
Salman	2024-04-01	900.00
Simran	2024-04-12	600.00

WHERE CLAUSE:

The WHERE clause is used in SQL to filter rows from a table based on a condition. Only the rows that satisfies the where condition are shown in the resultant table. It is optional part in the SQL query. If we don't write where condition, all the rows from the table are shown in the resultant table.

Syntax: SELECT column1, column2 FROM table_name [WHERE condition];

Example:

1. SELECT * FROM customers WHERE city **=** 'Pokhara';
2. SELECT * FROM customers WHERE city **!=** 'Pokhara';
3. SELECT * FROM orders WHERE amount **<** 500;
4. SELECT * FROM orders WHERE amount **<=** 500;
5. SELECT * FROM orders WHERE amount **>** 500;
6. SELECT * FROM orders WHERE amount **>=** 500;
7. SELECT * FROM customers WHERE city = 'Pokhara' **AND** name = 'Simran';
8. SELECT * FROM customers WHERE city = 'Pokhara' **OR** city = 'Butwal';
9. SELECT * FROM customers WHERE **NOT** city = 'Pokhara';
10. SELECT * FROM orders WHERE amount **BETWEEN** 300 **AND** 700;
11. SELECT * FROM orders WHERE amount **NOT BETWEEN** 300 **AND** 700;
12. SELECT * FROM customers WHERE city **IN** ('Pokhara', 'Butwal');
13. SELECT * FROM customers WHERE city **NOT IN** ('Pokhara', 'Butwal');
14. SELECT * FROM customers WHERE email **IS NULL**;
15. SELECT * FROM customers WHERE email **IS NOT NULL**;
16. SELECT * FROM customers WHERE name **LIKE** 'b%;

Like Operator:

SQL allows pattern matching with strings using LIKE operator. LIKE operator matches a string value against a pattern string containing wildcard characters. The wildcard characters for LIKE operator are underscore (_) and percent (%).

- '_' represents a single character.
- '%' represents zero characters or more characters.

Syntax:

```
SELECT column_names
FROM table_name
WHERE column_name LIKE pattern;
```

Pattern Examples:

1. 'A_' → Means any 2 character string starting with A.
2. '_200_' → Means any 4 character string with 200 in the middle.
3. 'A%' → Means any string starting with A.
4. '%200%' → Means any string that has 200 as its substring.

Order BY Clause:

The ORDER BY clause is used to sort the result of a query by the columns. By default, it sorts in ascending order (ASC). We can use **DESC** for descending order.

Example:

Ascending: SELECT * FROM customers ORDER BY name;

Descending: SELECT * FROM customers ORDER BY name DESC;

Sorting by multiple columns:

SELECT * FROM customers ORDER BY city, name;

LIMIT KEYWORD:

The LIMIT keyword is used to restrict the number of rows returned by a SQL query.

Example:

SELECT * FROM customers LIMIT 3; // Shows first 3 rows only

SELECT * FROM customers LIMIT 2 OFFSET 3;

// Skips first 3 rows and then shows 2 rows (4^{th} & 5^{th}) .

Alias:

An alias is a temporary name given to a column or table to make results easier to read or write shorter queries. We use **AS** keyword to give alias.

1. **Column Alias:**

```
SELECT name AS customer_name FROM
customers;
```

2. **Column Alias:**

```
SELECT c.name, o.amount
FROM customers AS c JOIN orders AS o
ON c.cid = o.cid;
```

Aggregate Functions:

Aggregate functions operate in group of values and return a single value. It is mainly used for analyzing, manipulating, and summarizing the data. The aggregate function supported by SQL are:

Function	Description
SUM (column)	Returns sum of all the values of specified column
AVG (column)	Returns average of all the values of specified column
COUNT (column)	Returns total number of NON-NULL values of specified column
COUNT (*)	Returns total number of rows in a table
MIN (column)	Returns lowest value among the values in the specified column
MAX (column)	Returns largest value among the values in the specified column

employee Table

id	name	salary	department
1	Aman	40000.00	HR
2	Riya	55000.00	IT
3	Sita	47000.00	Finance
4	John	60000.00	IT
5	Neha	52000.00	HR
6	Bob	*NULL*	Finance

1. **SUM (column):**

```
SELECT SUM(salary) AS total_salary FROM employees;
```

total_salary
254000.00

2. **AVG (column):**

```
SELECT AVG(salary) AS avg_salary FROM employees;
```

avg_salary
50800.000000

3. **COUNT (column):**

```
SELECT COUNT(salary) AS total_employees_with_salary FROM employees;
```

total_employees_with_salary
5

4. **COUNT (*):**

```
SELECT COUNT(*) AS total_rows FROM employees;
```

total_rows
6

5. **MIN (column):**

```
SELECT MIN(salary) AS min_salary FROM employees;
```

min_salary
40000.00

6. **MAX (column):**

```
SELECT MAX(salary) AS max_salary FROM employees;
```

max_salary
60000.00

Group BY:

The GROUP BY clause is used to **group rows** that have the **same value** in a column. It's typically used with **aggregate functions** (SUM, AVG, COUNT, etc.) to **summarize data** by group.

Syntax:

```
SELECT column, AGG_FUNCTION(column)
FROM table
GROUP BY column;
```

Example:

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department;
```

Employees Table:

id	name	salary	department
1	Aman	40000.00	HR
2	Riya	55000.00	IT
3	Sita	47000.00	Finance
4	John	60000.00	IT
5	Neha	52000.00	HR
6	Bob	*NULL*	Finance
7	Ramesh	30000.00	Sales
8	Suman	48000.00	Marketing
9	Nita	51000.00	HR
10	Anil	62000.00	IT

Output for Group by statement:

department	total_salary
Finance	47000.00
HR	143000.00
IT	177000.00
Marketing	48000.00
Sales	30000.00

Having clause:

The HAVING clause is like WHERE, but it is used to filter groups (not individual rows) after grouping is done.

Syntax:

```
SELECT column, AGG_FUNCTION(column)
FROM table
GROUP BY column
HAVING condition;
```

Example:

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 90000;
```

Output for Having statement:

department	total_salary
HR	143000.00
IT	177000.00

***

Unit 6

Graphics and Security

Embedding Images:

Embedding Images refers displaying images in the web pages. In PHP, we can dynamically embed images in the web pages using the given syntax:

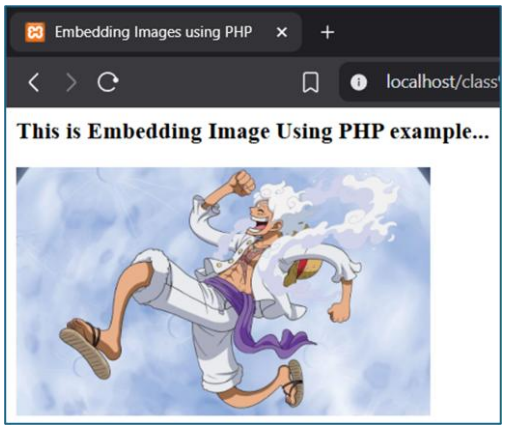
```
echo "<img src='image_url' height= '' alt='text_to_display_if_image_not_load'> ";
```

Example:

embedImage.php

```
<html>
    <head>
        <title>Embedding Images using PHP</title>
    </head>
    <body>
        <?php
            echo "<h3>This is Embedding Image Using PHP example...</h3>";
            echo "<img src='https://www.animationmagazine.net/wordpress/wp-content/uploads
                                /One-Piece-G5.jpg' height='200' alt='luffy-Image'>";
        ?>
    </body>
</html>
```

Output:



Pixel:

The smallest unit of digital image arranged in a grid is called pixel. Each pixel stores color and brightness information.

Images Types:

1. **JPEG**: Best for real-life photos. It supports many colors and loads quickly, but it does not support transparency.
2. **PNG:** deal for logos and clear images. It supports transparency and maintains high image quality, but the file size is bigger.
3. **GIF:** Great for simple animations. It supports limited colors (256) and basic transparency.

Embedding Buttons:

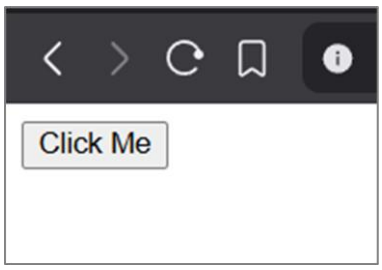
In PHP, we can dynamically embed buttons in the web pages.

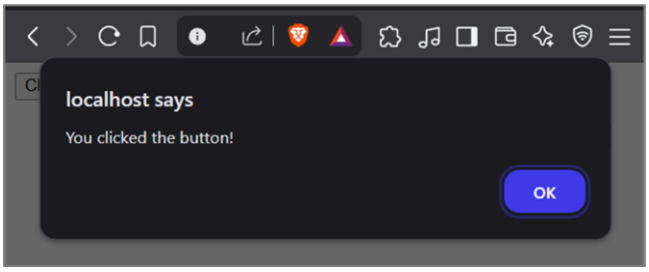
Example:

embedButton.php

```
<?php
    echo '<button onclick="alert(\'You clicked the button!\')"> Click Me </button>';
?>
```

Output:





Input Filtering:

Input Filtering is the process of validating user input to ensure it meets specified conditions before processing it. The goal of filtering is to make sure the data is valid, correct, and safe to use. The key aspects of Input filtering are Validating format, validating data types, validating range/ length. If the data is invalid (doesn't meet the validation criteria) the data is rejected. Some Examples of Input Filtering are Required Field Validation, Numeric Validation, Password Validation, Length Validation, Email Validation, etc.

(Yo validations haru ko code Unit 4 - form Processing ma xa)

Input Sanitizing:

Input sanitization is the process of cleaning up user input to ensure it's safe to use. The goal is to remove harmful content, like dangerous code or special characters, that could cause security issues, such as Cross-Site Scripting (XSS) or SQL Injection.

Example:

If a user types <script> alert('XSS attack'); </script> in a form input field, sanitization will change it to <script>alert('XSS attack');</script>.

So, it just shows as text and doesn't run any harmful code.

We can sanitize user input Using htmlspecialchars() function as:

sanitizeInput.php

```
<?php
    $user_input = "<script>alert('Hacked!')</script>";          # User Input Data from Form Field
    $safe_input = htmlspecialchars($user_input);                # Sanitizing user Input Data

    echo $user_input . "\n";            # Output: <script>alert('Hacked!')</script>
    echo $safe_input;                   # Output: &lt;script&gt;alert('Hacked!')&lt;/script&gt;
?>
```

Output:

```
<script>alert('Hacked!')</script>
&lt;script&gt;alert(&#039;Hacked!&#039;)&lt;/script&gt;
```

Input Filtering and Sanitizing are essential security practices in web development. Only filtering data or sanitizing data is not enough. We need to do both. We need both to:

- Prevent Injection attacks like SQL Injection, Cross-Site-Scripting (XSS), etc.
- Prevent system crash as invalid data (Eg: letters in number field) can cause exceptions or crashes.
- Preserve data Integrity by ensuring data in the database remains clean and consistent.
- Improves user experience by showing error messages early when the user enters invalid data.
- Filters and prevent malicious file names and contents from being processed.

Escape Output:

The process of changing special characters in user input into safe text so that they don't run as code when shown on a web page. Escape Output is done to prevent code from running and instead show it as normal text. Escape output is done before showing data on a web page while input sanitization is done before saving or processing the input.

Session Fixation:

Session Fixation is a type of attack where a hacker tricks a user into using a known or fixed session ID. Once the user logs in using that session ID, the attacker can hijack the session and gain access to the user's account.

How session Fixation Works:

1. Attacker creates a valid session ID.
2. Attacker tricks the user into using that session ID (via a link, form, etc.).
3. User logs in - but still using the attacker's session ID.
4. Attacker now uses the same ID to access the user's session.

We Prevent Session Fixation as:

- Always **generate a** new session ID after login.
- Use session_regenerate_id(true) in PHP to create a fresh ID.
- Set cookies to HttpOnly and Secure.
- Don't accept session IDs from URLs or hidden fields.

File Access:

In PHP, **file access** means reading from or writing to files stored on the server. PHP provides some built in functions to work with files.

1. **fopen():** To open a file for reading / writing.
2. **fread():** To read content from a file.
3. **fwrite():** To write content to a file.
4. **fclose():** To close the file.
5. **unlink():** To delete a file.

File Upload:

We need to care about following thing for secure file upload in PHP.

- Allow only specific file types (Ex: jpg, png, pdf).
- Limit file size to prevent overload (Ex: max 5MB).
- Rename uploaded files to prevent overwriting existing files.
- Set correct file permissions (Ex: read-only).
- Disable execution of scripts in the upload folder.
- Use .htaccess to block PHP/JS execution if needed.

Cross Site Scripting (XSS):

XSS (Cross-Site Scripting) is a web security vulnerability where an attacker injects malicious scripts into web pages. When other users open the page, the browser runs that script as if it came from the trusted website.

Why XSS is dangerous?

- It can steal session information.
- It can read private data entered by the user.
- It can send data to hacker's server.
- It can load malware on user's browser.
- It can perform actions (such as changing passwords or posting content) on its own.

We can prevent XSS by:

- Validating user Input
- Sanitizing user Input
- Set cookies to HttpOnly, etc.

Example:

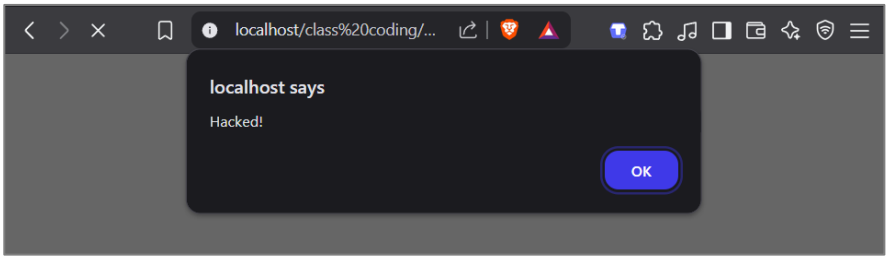
Suppose we are displaying comment in a page as:

```
echo $_POST['comment'];
```

When normal users submit comment as: 'Hello I am new user...' , following comment is shown in the web page:

Hello I am new user...

But when an attacker submit this type of comment: "<script>alert('Hacked!');</script>", this alert box is shown in other users browser.



***

Unit 7

Framework and CMS

Framework:

A set of reusable predefined libraries and components that provides a structure for application or web app development is called a framework. It saves time and effort by offering built-in tools, functions, and libraries so developers don't have to write everything from scratch.

Some PHP frameworks are:

1. **Laravel:**
   Most popular modern PHP framework with clean syntax and built-in tools like routing, authentication, and database handling.
2. **CodeIgniter:** Lightweight PHP framework that is easy to install and beginner friendly.
3. **Symfony:** A robust PHP framework used for complex enterprise-level applications.
4. **CakePHP:** Suitable for rapid application development.

MVC:

MVC stands for Model-View-Controller. It is a software architectural pattern that helps in organizing the application development process by dividing the application into 3 main components: Model, View, and Controller.

Features of MVC are:

- MVC separates the application into three components: Model, View, and Controller.
- It promotes reusability by allowing models and views to be used with different controllers.
- MVC improves modularity, i.e. we can modify or replace any component independently.
- It enhances maintainability by keeping business logic, UI, and control separate.
- MVC supports scalability, making it suitable for larger and more complex applications.
- It improves testability by allowing each component to be tested separately.

Components of MVC:

The components of MVC are given below:

1. Model (related to Database)
2. View (related to Front-end UI)
3. Controller (Bridge between Model & View - handles logic, decisions, and what to do next)

Model:

The Model in MVC represents the data and business logic of the application. It is responsible for interacting with the database – retrieve, store, and manipulate data. The Model does not handle user interface (UI). Model contains database interaction code, validation code, etc. The responsibilities of Model are:

- It Inserts, retrieves, updates and deletes data to/from database.
- It process data (i.e. perform calculations and validations).
- It maintains data consistency in the application.
- It notifies the controller of any changes in data that might require a UI update.

View:
The view in MVC represents User Interface (UI) of the application. It is responsible for displaying data in user-friendly way. It contains HTML code, CSS code, JS code, PHP (if required). It doesn't interact with database directly. The responsibilities of view are:

- It represents the User Interface (UI).
- It displays the data passed from the MODEL or CONTROLLER in a user-friendly format.
- It notifies the controller about actions like form submissions for further handling.
- It may also include some presentation logic – like which section of UI to show or hide based on data or user interactions.

Controller:
The Controller in MVC acts as a bridge between the Model and the View. It handles user input, processes it (if needed), and then calls the appropriate functions from the Model to retrieve or modify data. After that, it passes the data to the View to display the output. The responsibilities of controller are:

- **Handle User Requests** – Receives and processes User input data.
- **Call Model Methods** – Interacts with the Model to get or modify database data.
- **Update the View** – Sends data to the View for display.
- **Route Requests** – Determines which function to run based on the request.
- **Manage Authentication** – Checks user permissions for access control.
- **Handle Sessions** – Manages login status and session data.

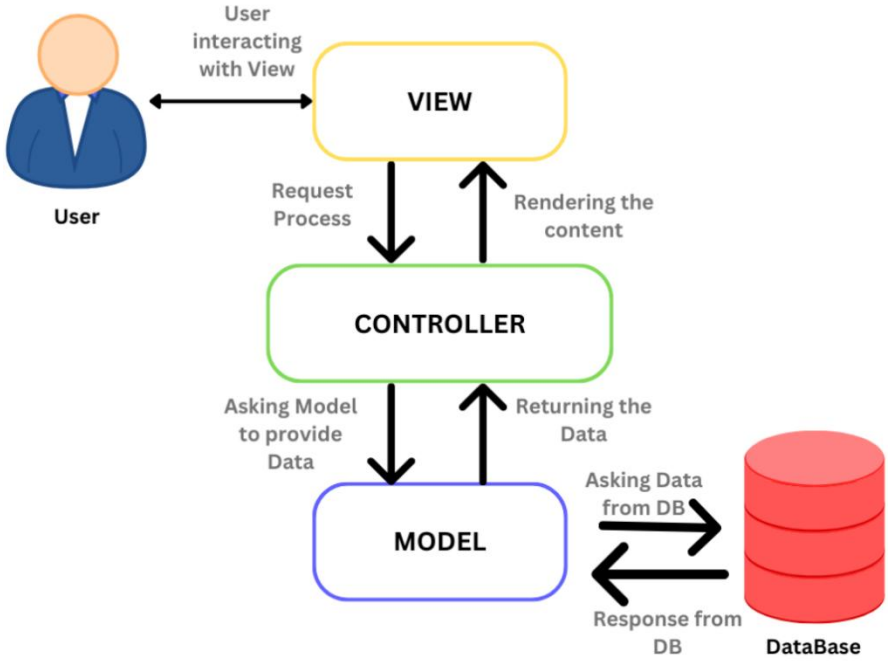


Fig: MVC architecture

Note:
Laravel (PHP), CodeIgniter (PHP), CakePHP (PHP), Spring (Java), Django (Python), etc. are MVC architecture-based Frameworks.

What is a CMS? **[CMS – Content Management System]**

A CMS is a software or tool that lets you create, manage, and modify website content without needing to write code from scratch. Some common CMS are:

1. WordPress: Most popular CMS for blogs and websites.
2. Joomla: Flexible and powerful for larger websites.
3. Drupal: Advanced, secure, used for complex projects.
4. Magento: CMS for e-commerce websites.

What can you do with a CMS?

We can do the following things with a CMS.

- Add/ edit / delete pages, blogs, images, or products.
- Use themes and plugins to customize design and features.
- Manage users, comments, and menus easily.
- Schedule content for future publishing.
- Track changes and version history.

Why use a CMS?

People use CMS because of following reasons:

- No deep coding required – Anyone can build and manage a site without programming knowledge.
- Saves time in development and updates – Quickly update content or design with just few clicks.
- Idel for non-developers as they can easily add, update, or delete content.
- Supports multiple users roles: Admin, editors, viewer, etc.
- Large community and support – popular CMS have plenty of tutorial, forums, and ready-made-tools.

Why not use a CMS?

People don't use CMS because of following reasons:

- CMS offers limited flexibility for custom features.
- CMS are slower than a custom-built sites due to unnecessary features or bloated plugins.
- Popular CMS are frequently targeted by hackers. Outdated plugins or themes can make a site vulnerable.
- Understanding CMS basics is easy but understanding advanced settings, SEO tools, etc., can be challenging.
- Requires regular updates of the core system, plugins, and themes to avoid bugs and security issues.

WordPress:

WordPress is a free and open-source Content Management System (CMS) used to create and manage websites without needing to write much code. It is the most popular CMS in the world. With WordPress, we can easily create blogs, business sites, portfolios, and online stores. WordPress offers thousands of themes and plugins. WordPress sites are SEO friendly and Mobile responsive.

Domain Name:

A domain name is the unique address used to access a website on the internet. Facebook.com, Instagram.com, etc. are the example of domain names.

Hosting Options:

Hosting is a service that makes an applications or website accessible on the internet. The different hosting options are:

1. **Shared Hosting:**
   Shared hosting is an affordable option where multiple websites share the same server. It's great for small sites and blogs, easy to set up, but offers limited resources and customization.
2. **Virtual Private Server (VPS) Hosting:**
   A physical server is divided into multiple **Virtual Private Servers (VPS)**, each with its own dedicated resources. It offers better performance and control than shared hosting, making it ideal for medium to high-traffic websites. While VPS hosting is scalable and more powerful, it costs more than shared hosting and requires some technical knowledge to manage.
3. **Dedicated Hosting:**
   Here, Entire server is dedicated to one website, offering full control over all its resources. It's ideal for large websites with high traffic or complex needs like e-commerce. It provides top performance and reliability but is costly and requires advanced technical skills to manage.
4. **Cloud Hosting:**
   Cloud hosting uses cloud computing technology is used to host websites. It allows resources to scale up or down easily based on demand. Ideal for sites with varying traffic, it offers flexibility, high uptime, and pay-as-you-go pricing.

Dashboard:

A dashboard is a visual interface that displays important information and controls in one place, like a control panel. It helps users quickly access data, statistics, or tools.

Pages:

Pages refers to different sections of a website that users can navigate.
Ex: Home page, About page, Contact page, Services Page, Login / Sign-up page, etc.,

Directory Permissions:

Directory permissions control who can access, read, write, or execute files and folders on a server. They are crucial for security and functionality in hosting environments.

***